



Control Command Guide

1.3.8.0

IPEXAR-UIC



Table of Contents

1 Introduction	14
1.1 License Requirements	14
1.2 Client Connection and API Commands	14
1.3 HTTP Requests	14
2 Command Overview	15
3 Command set	16
3.1 Command set listener	16
3.1.1 Method	16
3.1.2 Description	16
3.1.3 Arguments	16
3.1.4 Notes	16
3.1.5 Response	16
3.1.6 Examples	16
3.1.7 Response Examples	17
3.1.8 Global Caché GC-100 Series IR/Sensor Connector Wiring	18
3.1.9 Global Caché iTach and GlobalConnect Series IR/Sensor Connector Wiring	18
3.1.10 Global Caché Flex Link Relay & Sensor Cable Wiring	18
3.1.11 Global Caché Technical Notes	19
3.1.12 Global Caché Example	20
3.2 Command set events	21
3.2.1 Method	21
3.2.2 Description	21
3.2.3 Arguments	21
3.2.4 Notes	21
3.2.5 Response	21
3.2.6 Example	21
3.2.7 Response Example	21
3.3 Command set var <i>*Updated</i>	22
3.3.1 Method	22
3.3.2 Description	22
3.3.3 Arguments	22
3.3.4 Notes	22
3.3.5 Response	22
3.3.6 Examples	22
3.3.7 Preset Examples	22
3.3.8 Response Examples	22
3.4 Command set ui	23
3.4.1 Method	23
3.4.2 Description	23
3.4.3 Arguments	23
3.4.4 Notes	23
3.4.5 Response	23
3.4.6 Examples	23
3.4.7 Preset Examples	23
3.4.8 Response Examples	23
3.5 Command set ui_button <i>(future functionality pending)</i>	24
3.5.1 Method	24
3.5.2 Description	24
3.5.3 Arguments	24
3.5.4 Notes	25
3.5.5 Response	26
3.5.6 Examples	27
3.5.7 Preset Examples	27
3.5.8 Response Examples	27

Table of Contents continued...

3.6 Command set ui_label	28
3.6.1 Method	28
3.6.2 Description	28
3.6.3 Arguments	28
3.6.4 Notes	28
3.6.5 Response	28
3.6.6 Examples	28
3.6.7 Preset Examples	29
3.6.8 Response Examples	29
3.7 Command set ui_image	30
3.7.1 Method	30
3.7.2 Description	30
3.7.3 Arguments	30
3.7.4 Notes	30
3.7.5 Response	30
3.7.6 Examples	30
3.7.7 Preset Examples	30
3.7.8 Response Examples	30
3.8 Command set ui_page <i>*updated</i>	31
3.8.1 Method	31
3.8.2 Description	31
3.8.3 Arguments	31
3.8.4 Notes	31
3.8.5 Response	31
3.8.6 Examples	31
3.8.7 Preset Examples	31
3.8.8 Response Examples	31
3.9 Command set ui_indicator	32
3.9.1 Method	32
3.9.2 Description	32
3.9.3 Arguments	32
3.9.4 Notes	32
3.9.5 Response	32
3.9.6 Examples	32
3.9.7 Preset Examples	32
3.9.8 Response Examples	32
3.10 Command set ui_slider	33
3.10.1 Method	33
3.10.2 Description	33
3.10.3 Arguments	33
3.10.4 Notes	33
3.10.5 Response	33
3.10.6 Examples	33
3.10.7 Preset Examples	33
3.10.8 Response Examples	34
3.11 Command set ui_redirect	35
3.11.1 Method	35
3.11.2 Description	35
3.11.3 Arguments	35
3.11.4 Notes	35
3.11.5 Response	35
3.11.6 Examples	35
3.11.7 Preset Examples	35
3.11.8 Response Examples	35

Table of Contents continued...

3.12 Command set ui_revert	36
3.12.1 Method	36
3.12.2 Description	36
3.12.3 Arguments	36
3.12.4 Notes	36
3.12.5 Response	36
3.12.6 Examples	36
3.12.7 Preset Examples	36
3.12.8 Response Examples	36

Table of Contents continued...

4 Command get	37
4.1 Command get api	37
4.1.1 Method	37
4.1.2 Description	37
4.1.3 Arguments	37
4.1.4 Notes	37
4.1.5 Response	37
4.1.6 Examples	37
4.1.7 Response Examples	37
4.2 Command get events	38
4.2.1 Method	38
4.2.2 Description	38
4.2.3 Arguments	38
4.2.4 Notes	38
4.2.5 Response	38
4.2.6 Examples	38
4.2.7 Response Examples	38
4.3 Command get var	39
4.3.1 Method	39
4.3.2 Description	39
4.3.3 Arguments	39
4.3.4 Notes	39
4.3.5 Response	39
4.3.6 Examples	39
4.3.7 Preset Examples	39
4.3.8 Response Examples	39
4.4 Command get ui	40
4.4.1 Method	40
4.4.2 Description	40
4.4.3 Arguments	40
4.4.4 Notes	40
4.4.5 Response	40
4.4.6 Examples	40
4.4.7 Preset Examples	40
4.4.8 Response Examples	40
4.5 Command get ui_button <i>*updated (future functionality pending)</i>	41
4.5.1 Method	41
4.5.2 Description	41
4.5.3 Arguments	41
4.5.4 Notes	41
4.5.5 Response	41
4.5.6 Examples	41
4.5.7 Preset Examples	42
4.5.8 Response Examples	42
4.6 Command get ui_indicator	43
4.6.1 Method	43
4.6.2 Description	43
4.6.3 Arguments	43
4.6.4 Notes	43
4.6.5 Response	43
4.6.6 Examples	43
4.6.7 Preset Examples	43
4.6.8 Response Examples	43

Table of Contents continued...

4.7 Command get ui_slider	44
4.7.1 Method	44
4.7.2 Description	44
4.7.3 Arguments	44
4.7.4 Notes	44
4.7.5 Response	44
4.7.6 Examples	44
4.7.7 Preset Examples	44
4.7.8 Response Examples	44
4.8 Command get ui_page <i>*new</i>	45
4.8.1 Method	45
4.8.2 Description	45
4.8.3 Arguments	45
4.8.4 Notes	45
4.8.5 Response	45
4.8.6 Examples	45
4.8.7 Preset Examples	45
4.8.8 Response Examples	45

Table of Contents continued...

5 Command send	46
5.1 Command send gc *Licensed feature (future functionality pending)	53
5.1.1 Method	53
5.1.2 Description	53
5.1.3 Arguments	53
5.1.4 Notes	53
5.1.5 Response	54
5.1.6 Examples	54
5.1.7 Preset Examples	55
5.1.8 Response Examples	55
5.2 Command send tcp (future functionality pending)	56
5.2.1 Method	56
5.2.2 Description	56
5.2.3 Arguments	56
5.2.4 Notes	56
5.2.5 Response	57
5.2.6 Examples	57
5.2.7 Preset Examples	58
5.2.8 Response Examples	58
5.3 Command send telnet (future functionality pending)	59
5.3.1 Method	59
5.3.2 Description	59
5.3.3 Arguments	59
5.3.4 Notes	59
5.3.5 Response	61
5.3.6 Examples	61
5.3.7 Preset Examples	62
5.3.8 Response Examples	62
5.4 Command send udp (future functionality pending)	63
5.4.1 Method	63
5.4.2 Description	63
5.4.3 Arguments	63
5.4.4 Notes	63
5.4.5 Response	64
5.4.6 Examples	64
5.4.7 Preset Examples	65
5.4.8 Response Examples	65
5.5 Command send http (future functionality pending)	66
5.5.1 Method	66
5.5.2 Description	66
5.5.3 Arguments	66
5.5.4 Notes	66
5.5.5 Response	67
5.5.6 Examples	67
5.5.7 Preset Examples	69
5.5.8 Response Examples	69
5.6 Command send notification (pending future command)	70
5.6.1 Method	70
5.6.2 Description	70
5.6.3 Arguments	70
5.6.4 Notes	70
5.6.5 Response	70
5.6.6 Examples	70
5.6.7 Preset Examples	70
5.6.8 Response Examples	70

Table of Contents continued...

6 Command preset	71
6.1 Command preset add	71
6.1.1 Method	71
6.1.2 Description	71
6.1.3 Arguments	71
6.1.4 Notes	71
6.1.5 Response	71
6.1.6 Examples	71
6.1.7 Response Examples	71
6.2 Command preset delay	72
6.2.1 Method	72
6.2.2 Description	72
6.2.3 Arguments	72
6.2.4 Notes	72
6.2.5 Response	72
6.2.6 Examples	72
6.2.7 Response Examples	72
6.3 Command preset delete	73
6.3.1 Method	73
6.3.2 Description	73
6.3.3 Arguments	73
6.3.4 Notes	73
6.3.5 Response	73
6.3.6 Examples	73
6.3.7 Response Examples	73
6.4 Command preset load	74
6.4.1 Method	74
6.4.2 Description	74
6.4.3 Arguments	74
6.4.4 Notes	74
6.4.5 Response	74
6.4.6 Examples	74
6.4.7 Response Examples	74

Table of Contents continued...

7 Preset Logic Commands	75
7.1 Preset Command instr()	76
7.1.1 Method	76
7.1.2 Description	76
7.1.3 Arguments	76
7.1.4 Notes	76
7.1.5 Response	76
7.1.6 Examples	76
7.2 Preset Command instrrev()	77
7.2.1 Method	77
7.2.2 Description	77
7.2.3 Arguments	77
7.2.4 Notes	77
7.2.5 Response	77
7.2.6 Examples	77
7.3 Preset Command replace()	79
7.3.1 Method	79
7.3.2 Description	79
7.3.3 Arguments	79
7.3.4 Notes	79
7.3.5 Response	79
7.3.6 Examples	79
7.4 Preset Command strlen()	80
7.4.1 Method	80
7.4.2 Description	80
7.4.3 Arguments	80
7.4.4 Notes	80
7.4.5 Response	80
7.4.6 Examples	80
7.5 Preset Command substr()	81
7.5.1 Method	81
7.5.2 Description	81
7.5.3 Arguments	81
7.5.4 Notes	81
7.5.5 Response	81
7.5.6 Examples	81
7.6 Preset Command substring()	82
7.6.1 Method	82
7.6.2 Description	82
7.6.3 Arguments	82
7.6.4 Notes	82
7.6.5 Response	82
7.6.6 Examples	83
7.7 Preset Command trim()	84
7.7.1 Method	84
7.7.2 Description	84
7.7.3 Arguments	84
7.7.4 Notes	84
7.7.5 Response	84
7.7.6 Examples	84
7.8 Preset Command val()	85
7.8.1 Method	85
7.8.2 Description	85
7.8.3 Arguments	85
7.8.4 Notes	85
7.8.5 Response	85
7.8.6 Examples	86

Table of Contents continued...

7.9 Preset Command dec()	87
7.9.1 Method	87
7.9.2 Description	87
7.9.3 Arguments	87
7.9.4 Notes	87
7.9.5 Response	87
7.9.6 Examples	87
7.10 Preset Command Inc()	88
7.10.1 Method	88
7.10.2 Description	88
7.10.3 Arguments	88
7.10.4 Notes	88
7.10.5 Response	88
7.10.6 Examples	88
7.11 Preset Command loop while() (pending future command)	89
7.11.1 Method	89
7.11.2 Description	89
7.11.3 Arguments	89
7.11.4 Notes	89
7.11.5 Response	89
7.11.6 Examples	89
7.12 Preset Command select case()	90
7.12.1 Method	90
7.12.2 Description	90
7.12.3 Arguments	90
7.12.4 Notes	90
7.12.5 Response	90
7.12.6 Examples	91
7.13 Preset Command return()	93
7.13.1 Method	93
7.13.2 Description	93
7.13.3 Arguments	93
7.13.4 Notes	93
7.13.5 Response	93
7.13.6 Examples	93
7.14 Preset Command suspend()	94
7.14.1 Method	94
7.14.2 Description	94
7.14.3 Arguments	94
7.14.4 Notes	94
7.14.5 Response	94
7.14.6 Examples	94

Table of Contents continued...

8 Preset Operators	95
8.1 Preset Operator 'And' &	95
8.1.1 Method	95
8.1.2 Description	95
8.1.3 Examples	95
8.2 Preset Operator 'AndAlso' &&	95
8.2.1 Method	95
8.2.2 Description	95
8.2.3 Examples	95
8.3 Preset Operator 'OrElse'	96
8.3.1 Method	96
8.3.2 Description	96
8.3.3 Examples	96
8.4 Preset Operator 'EqualTo' ==	96
8.4.1 Method	96
8.4.2 Description	96
8.4.3 Examples	96
8.5 Preset Operator 'NotEqualTo' !=	96
8.5.1 Method	96
8.5.2 Description	96
8.5.3 Examples	96
8.6 Preset Operator 'LessThan' <	97
8.6.1 Method	97
8.6.2 Description	97
8.6.3 Examples	97
8.7 Preset Operator 'GreaterThan' >	97
8.7.1 Method	97
8.7.2 Description	97
8.7.3 Examples	97
8.8 Preset Operator 'LessThanOrEqualTo' <=	98
8.8.1 Method	98
8.8.2 Description	98
8.8.3 Examples	98
8.9 Preset Operator 'GreaterThanOrEqualTo' >=	98
8.9.1 Method	98
8.9.2 Description	98
8.9.3 Examples	98
8.10 Preset Operator 'Not' not	98
8.10.1 Method	98
8.10.2 Description	98
8.10.3 Examples	98

Table of Contents continued...

9 Preset System Variables	99
9.1 <<ui_name>>	100
9.1.1 Method	100
9.1.2 Description	100
9.1.3 Notes	100
9.1.4 Response	100
9.1.5 Examples	100
9.2 <<button_name>>	101
9.2.1 Method	101
9.2.2 Description	101
9.2.3 Notes	101
9.2.4 Response Examples	101
9.2.5 Examples	102
9.3 <<button_position>>	103
9.3.1 Method	103
9.3.2 Description	103
9.3.3 Notes	103
9.3.4 Response	103
9.3.5 Examples	103
9.4 <<slider_name>>	104
9.4.1 Method	104
9.4.2 Description	104
9.4.3 Notes	104
9.4.4 Response	104
9.4.5 Examples	105
9.5 <<slider_value>>	106
9.5.1 Method	106
9.5.2 Description	106
9.5.3 Notes	106
9.5.4 Response	106
9.5.5 Examples	106
9.6 <<pin_code>> (pending future variable)	107
9.6.1 Method	107
9.6.2 Description	107
9.6.3 Notes	107
9.6.4 Response	107
9.6.5 Examples	107
9.7 <<encoder_name>>	108
9.7.1 Method	108
9.7.2 Description	108
9.7.3 Notes	108
9.7.4 Response	108
9.7.5 Examples	108
9.8 <<decoder_name>>	109
9.8.1 Method	109
9.8.2 Description	109
9.8.3 Notes	109
9.8.4 Response	109
9.8.5 Examples	109
9.9 <<send_response>> (pending future variable)	110
9.9.1 Method	110
9.9.2 Description	110
9.9.3 Notes	110
9.9.4 Response	110
9.9.5 Examples	110

Table of Contents continued...

COMMAND GUIDE V1.3.8.0

Appendix A – Preset Logic

111

Command results

115

Preset results

122

- Preset result examples of default results
- Preset result examples using return(true)
- Preset result examples using return(false)
- Preset result examples using return(null)
- Preset result examples using return(true,true)
- Preset result examples using return(false,true)
- Preset result examples using return(null,true)

123

124

125

126

127

129

130

Logic Examples

132

Appendix B – UI Snippets

136

- Login Codes
- Phone Dialer

136

141

Appendix C – How to HTTP request

154

Appendix D – Definition of terms

156

Appendix E – *Breaking* Changes

156

COMMAND GUIDE V1.3.8.0

1 Introduction

This document describes everything that a developer needs to be aware of to use the ARRANGER Control UI command guide and develop client control applications for devices.

1.1 License Requirements

The Controller must have a valid license key entered before use or connecting to the TCP control port 6980. If no valid license is active the Controller will return **Invalid license** and terminate the TCP connection. Contact your local distributor for licencing information.

1.2 Client Connection

Third party controllers connect to the Arranger Controller and issue commands using ASCII strings terminated with a carriage return <cr> (0x0D). This allows any TCP client to be used with the system.

The Arranger Controller listens on TCP port 6980. Once a successful TCP connection is established a welcome message **Connection Successful** will be sent.

A constant TCP connection to the Arranger Controller is recommended to maintain status changes of the system from notification events.

An optional security key can be used with all TCP API commands made to port 6980. The keyword **key**: along with the security key are added to the API command before any parameters.

1.3 HTTP requests

It is also possible to control the system with HTTP GET and POST requests.

A security key must be sent with any request. Security keys are generated by admin level users on the Global Settings / Security Key tab.

GET = `http://<controllerURL>/api/command/<CONTROLLER_API_COMMAND>/<KEY>`

POST = `http://<controllerURL>/api/command{"cmd":"<CONTROLLER_API_COMMAND>","key":"<KEY>"}`

Refer Appendix C - How to HTTP request

COMMAND GUIDE V1.3.8.0**2 Command Overview**

Commands are in a simple ASCII text format which is not case sensitive. For each command, the Arranger Controller responds with a response which contains the return status (i.e. whether the command succeeded or not) and, if successful, the Response of a get command. The API is to be used with synchronous communication.

- All commands and response are terminated with a carriage return <cr> \x0D
 - Commands are not case sensitive
 - User Interface and Element names are limited to 20 characters
 - Invalid commands will return **error [Unknown]<cr>**
 - Missing security key will return **error [security key missing]<cr>**
 - Invalid security key will return **error [security key invalid]<cr>**
-
- send gc 169.254.1.70 4998 setstate,1:1,1<cr>
 - send tcp 10.1.1.10 1234 "command"<cr>
 - set ui_button myUI myBtn position down<cr>

COMMAND GUIDE V1.3.8.0

3 Command set

The **set** commands are used to change various conditions.

3.1 Command set listener

3.1.1 Method

```
set listener [key:<security_key>] <notify_ip> <notify_port> <protocol> <device_ip> <condition> <state> ...  
<device_io> <preset_name> [<delay>]<cr>
```

3.1.2 Description

The command **set listener** utilises Global Caché device sensor notify functionality. This enables a preset to be triggered from a sensor notify beacon sent when a Global Caché device's input status changes from a switch or PIR. This command provides the same functionality as the **Listeners** tab in the UI.

3.1.3 Arguments

argument	type	contains
<notify_ip>	string	239.255.250.250
<notify_port>	integer	9160
<protocol>	string	UDP
<device_ip>	string	IP Address of the device
<condition>	string	on off any
<state>	string	enabled disabled
<device_io>	integer	Physical Global Caché device input sensor port 1 to 6
<preset_name>	string	Name of the preset to be executed
<delay>	integer	Optional delay time of preset execution from 0 to 60 minutes

3.1.4 Notes

- Supported Global Caché devices: IP2IR, WF2IR, GCIR3, Flex Link Relay & Sensor Cable.
- <condition>** input state as **on** (closed contact) | **off** (open contact) | **any** (contact closing or opening).
- <state>** is the running state of the set listener service as **enabled** or **disabled**.
- Optional **<delay>** is used to delay the preset for the specified amount of time in minutes which is reset each time the command is received.

3.1.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	listener	<space>	<i>success / error [message]</i>	<cr>

3.1.6 Examples

```
set listener 239.255.250.250 9160 udp 10.1.1.10 on enabled 1 preset1<cr>  
set listener 239.255.250.250 9160 udp 10.1.1.10 off enabled 2 preset2<cr>  
set listener 239.255.250.250 9160 udp 10.1.1.10 any enabled 3 preset3<cr>  
set listener 239.255.250.250 9160 udp 10.1.1.10 on enabled 1 preset4 30<cr>  
set listener 239.255.250.250 9160 udp 10.1.1.10 on disabled 1<cr>  
set listener 239.255.250.250 9160 udp 10.1.1.10 off disabled 2<cr>  
set listener 239.255.250.250 9160 udp 10.1.1.10 any disabled 3<cr>  
set listener key:abc123 239.255.250.250 9160 udp 10.1.1.10 on enabled 1 preset1<cr>
```

ToC

3.1.7 Response Examples

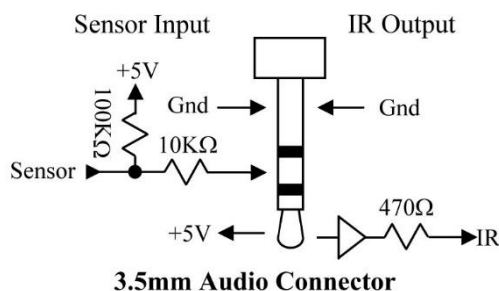
```
set listener success<cr>

set listener error [incomplete]<cr>
set listener error [duplicate]<cr>
set listener error [no active listener found]<cr>
set listener error [invalid notify ip]<cr>
set listener error [invalid notify port]<cr>
set listener error [reserved notify port]<cr>
set listener error [invalid protocol]<cr>
set listener error [invalid device ip]<cr>
set listener error [invalid condition]<cr>
set listener error [invalid state]<cr>
set listener error [invalid device io]<cr>
set listener error [preset '<preset_name>' not found]<cr>
set listener error [invalid delay]<cr>
```

COMMAND GUIDE V1.3.8.0

3.1.8 Global Caché GC-100 Series

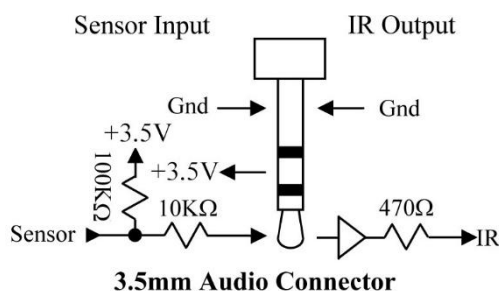
IR/Sensor Connector Wiring



Note: Use connector RING and SLEEVE for closed contact detection

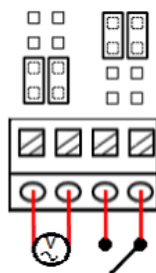
GC-100 devices are not currently supported for set listener

3.1.9 Global Caché iTach and GlobalConnect Series IR/Sensor Connector Wiring



Note: Use connector TIP and SLEEVE for closed contact detection

3.1.10 Global Caché Flex Link Relay & Sensor Cable Wiring



Note: Check jumper positions for either voltage or closed contact detection

3.1.11 Global Caché Technical Notes**Global Caché sensor cables**

GC-100 series

GC-SV1 Video Out Sensor

GC-SP1 AC/DC Voltage Sensor

GC-SC1 Contact Closure Sensor *

IP2IR, WF2IR and GCIR3

IT-SV1 Video Out Sensor

IT-SP1 AC/DC Voltage Sensor

IT-SC1 Contact Closure Sensor *

Flex series

Not required

* For closed contact detection a Global Caché GC-SC1 or IT-SC1 Contact Closure Sensor cable is not really required if you can make your own. With a 3.5mm stereo phono connector wired with tip and sleeve or ring and sleeve depending on the model as described above in 3.1.8 and 3.1.9.

Global Caché IR outputs and sensor inputs share a common connector and LED indicator. Each I/O connector is independently configured using the assistant. Each I/O connector has three contacts configured as either an infrared (IR) output or sensor input.

When configured as an output, the indicator will blink as an IR command is transmitted. When functioning as an input, the indicator is on when a positive input or no connection is present. The maximum sensor input voltage is $\pm 24V$, with an on indication for voltages greater than 2.5V and off when less than 0.8V with an input impedance of $\sim 100K\Omega$.

3.1.12 Global Caché Example

Here is a simple example of a system using a Global Caché iTach IP2IR connected to a push button to activate a Global Caché iTach IP2CC relay.



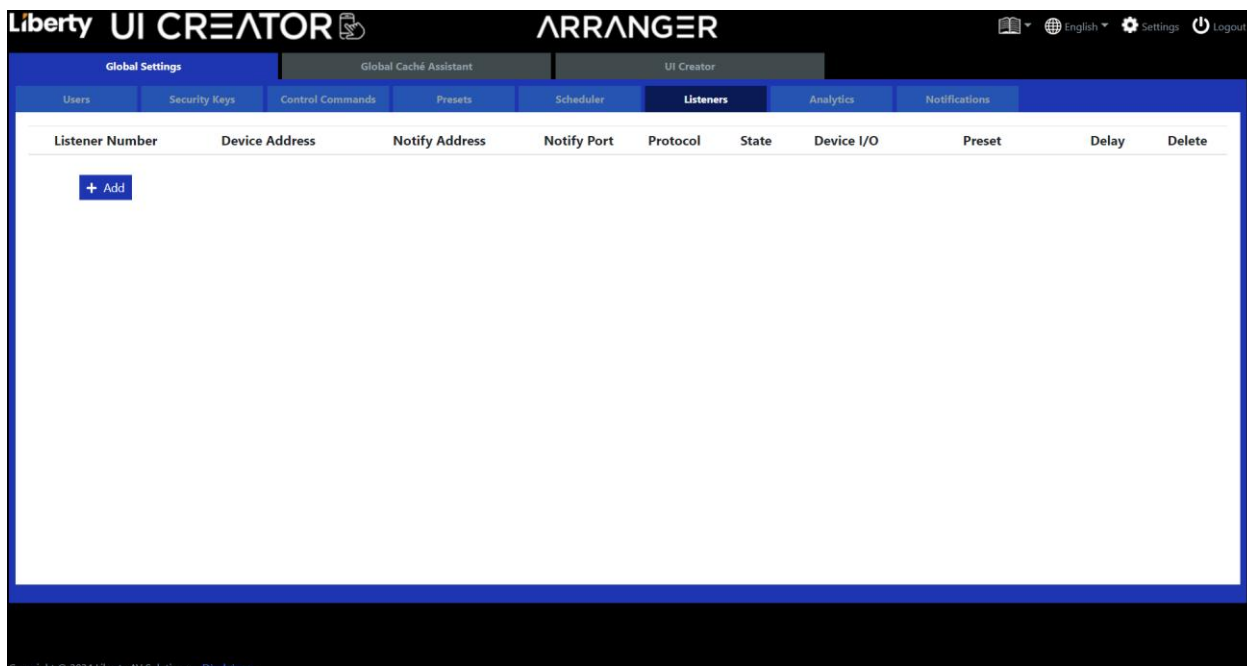
The command to establish the listener:

```
set listener 239.255.250.250 9160 udp 172.30.10.108 on enabled 1 triggerPreset
```

Preset triggerPreset would contain the following:

```
send gc 169.254.1.70 4998 setstate,1:1,1  
preset delay 1000  
send gc 169.254.1.70 4998 setstate,1:1,0
```

The UI Listeners tab will reflect set listener commands.



COMMAND GUIDE V1.3.8.0

3.2 Command set events

3.2.1 Method

```
set events [key:<security_key>] <event_name> <function> <value><cr>
```

3.2.2 Description

The command **set events** is used to **enable** or **disable** events created on the Scheduler tab.

3.2.3 Arguments

argument	type	contains
<event_name>	string	Name of the event
<function>	string	state
<value>	string	enabled disabled

3.2.4 Notes

- Event must be created from the Events tab before using command.
- Refer corresponding **get events** command.

3.2.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	events	<space>	<i>success / error [message]</i>	<cr>

3.2.6 Examples

```
set events myEvent state enabled<cr>
set events myEvent state disabled<cr>
set events key:abc123 myEvent state enabled<cr>
```

3.2.7 Response Examples

```
set events state success<cr>

set events error [incomplete]<cr>
set events error [event '<event_name>' not found]<cr>
set events error [invalid parameter]<cr>
set events error [invalid value '<value>']<cr>
```

COMMAND GUIDE V1.3.8.0

3.3 Command set var

3.3.1 Method

```
set var [key:<security_key>] <var_name> <value> [save]<cr>
```

3.3.2 Description

The command **set var** is used to store any user defined variable values including UI element names. The **save** option allows the variable to be saved as a file where data is permanently maintained until deleted.

3.3.3 Arguments

argument	type	contains
<var_name>	string	Name of the variable
<value>		string integer boolean [delete]
string	string	"any length string wrapped in quotes"
integer	integer	Any integer value
boolean	string	true false
[delete]	string	Keyword to delete variable
save	string	Keyword to save the variable to file

3.3.4 Notes

- <var_name> variable name up to 256 characters.
- Using keyword **save** will store the variable as a file.
- Refer **Appendix A** – Preset Logic
- Refer corresponding **get var** command.

3.3.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	var	<space>	<i>success / error [message]</i>	<cr>

3.3.6 Examples

```
set var myVar true<cr>
set var myVar false<cr>
set var myVar "any string with no practical length limit"<cr>
set var myVar 100 save<cr>
set var myVar [delete]<cr>
set var key:abc123 myVar true<cr>
```

3.3.7 Preset Examples

```
set var mySliderValue <<slider_value>> save
set var myBtnPosition <<button_position>>
set var myElement "Button01"
```

3.3.8 Response Examples

```
set var success<cr>

set var error [incomplete]<cr>
set var error [var_name max 256 characters]<cr>
```

COMMAND GUIDE V1.3.8.0

3.4 Command set ui

3.4.1 Method

```
set ui [key:<security_key>] <ui_name> <service> [timeout <timeout>] [clients <clients>] [login <pin>]<cr>
```

3.4.2 Description

The command **set ui** is used to **enable**, **disable** or **logout** a User Interface service. The UI login type and **pin** number, **client** limit and **timeout** can also be set.

3.4.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<service>	string	enabled disabled logout
timeout <timeout>	integer	Optional to set a client side timeout from 5 to 240 minuets
clients <clients>	integer	Optional to set the maximum client limit from 1 to 100
login <pin>	integer	Optional to set a fixed or random 4 digit login pin code

3.4.4 Notes

- Service **logout** is the same as **disabled** then **enabled**.
- The service must be enabled for **timeout**, **clients** and **login** options.
- Within a preset <<ui_name>> can replace <ui_name>.
- Refer corresponding **get ui** command.

3.4.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui	<space>	<i>success / error [message]</i>	<cr>

3.4.6 Examples

```
set ui myUI disabled<cr>
set ui myUI logout<cr>
set ui myUI enabled<cr>
set ui myUI enabled clients 10 login 1234<cr>
set ui myUI logout<cr>
set ui myUI enabled timeout 30 clients 10 login random<cr>
set ui key:abc123 myUI disabled<cr>
```

3.4.7 Preset Examples

```
set ui myUI logout
set ui <<ui_name>> logout
```

3.4.8 Response Examples

```
set ui success<cr>

set ui error [incomplete]<cr>
set ui error [ui '<ui_name>' not found]<cr>
set ui error [invalid clients '<value>']<cr>
set ui error [invalid pin '<value>']<cr>
set ui error [invalid timeout '<value>']<cr>
set ui error [invalid parameter]<cr>
```

COMMAND GUIDE V1.3.8.0

3.5 Command set ui_button (future functionality pending)

3.5.1 Method

```
set ui_button [key:<security_key>] <ui_name> <button_name> / <group_name> <function> ...  
[<button_position>] [<value>] [bypass] / [<time>] [apply] [glow] [<enc_btn_name>]<cr>
```

3.5.2 Description

The command **set ui_button** is used to control a button within an active User Interface.

3.5.3 Arguments

argument	type	contains	
<ui_name>	string	Name of the User Interface	
<button_name> / <group_name>	string	Name of the button or button group	
<function>	string	position state text press visibility image	
position	string	<button_position>	up down both clear group
state	string	<value>	enabled disabled revert
text	string	<value>	Object reference or a "wrapped string in quotation marks"
visibility	boolean	<value>	true false
image	string	<value>	original or image filename
bypass	string	Optional keyword to perform button actions except executing the preset	
<enc_btn_name>	string	Optional Split button Encoder button name used with <button_position> down to set the routing connection of the specified Decoder button, or use null to clear the Decoder button routing connection	
<time>	integer	Optional time with <function> state disabled to disable button for 0 to 180 seconds	
apply	string	Optional Keyword used with Toggle and Split buttons position to apply the preset on position change	
glow	string	Optional Keyword used with press to show button glow	

COMMAND GUIDE V1.3.8.0

3.5.4 Notes

- The UI service must be enabled.
- Within a preset <<button_name>> can replace <button_name> and <<ui_name>> can replace <ui_name>.
- Within a split button preset <<encoder_name>> and <<decoder_name>> can be used for button text <value>.
- For button and label **text**, \r can be used for a line break.
- For button **press**, optional keyword **bypass** is useful for status updates and not executing button presets.
- For button **press**, the pressed button must be enabled the same as a physical press.
- <function> **position** is only required for **Toggle**, **Radio Toggle** and **Split** button types.
With button groups only **position** and **state** functions are available. **Radio Toggle** buttons can only use **position up**.
- <position> **clear** is used only for Split Button Groups to reset their routing connections.
- <button_position> **group** is required when setting a split button groups routing connections from a get matrix command json return.
- <time> as a range of 0 to 180 seconds.
- Button groups can use state **revert** after **disable** to return the grouped buttons to their previous states.
- Split buttons refer to User Manual section 2.4.2.5 for further details.
- Refer corresponding **get ui_button** command.

• Momentary ○ state ○ text ○ press ○ visibility ○ image	• Toggle ○ state ○ text ○ press ○ position ○ visibility ○ image	• Radio Toggle ○ state ○ text ○ press ○ position ○ visibility ○ image	• Repeat ○ state ○ text ○ press ○ visibility ○ image	• Split ○ state ○ text ○ press ○ position ○ visibility ○ image	• QR Code ○ state ○ press ○ visibility
• Momentary Group ○ state ○ text ○ visibility	• Toggle Group ○ state ○ text ○ position ○ visibility	• Radio Toggle Group ○ state ○ text ○ position (up) ○ visibility	• Repeat Group ○ state ○ text ○ visibility	• Split Group ○ state ○ text ○ clear ○ position (up) ○ visibility	• QR Code Group ○ state ○ visibility

COMMAND GUIDE V1.3.8.0

3.5.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_button	<space>	success / error [message]	<cr>

3.5.6 Examples

```
set ui_button myUI myBtn position up<cr>
set ui_button myUI myBtn position down<cr>
set ui_button myUI myBtn position up apply<cr>
set ui_button myUI myBtn position down apply<cr>
set ui_button myUI myBtn state enabled<cr>
set ui_button myUI myBtn state disabled<cr>
set ui_button myUI myBtn state disabled 30<cr>
set ui_button myUI myBtn text up "myText"<cr>
set ui_button myUI myBtn text down "line1\rline2"<cr>
set ui_button myUI myBtn text both ""<cr>
set ui_button myUI myBtn text <button_position> text_with_no_spaces<cr>
set ui_button myUI myBtn visibility true<cr>
set ui_button myUI myBtn visibility false<cr>
set ui_button myUI myBtn image up "myImg.png"<cr>
set ui_button myUI myBtn image down "myImg.png"<cr>
set ui_button myUI myBtn image both "myImg.png"<cr>
set ui_button myUI myBtn image up original<cr>
set ui_button myUI myBtn press<cr>
set ui_button myUI myBtn press bypass<cr>
set ui_button myUI myBtn press glow<cr>
set ui_button myUI myBtnGroup position up<cr>
set ui_button myUI myBtnGroup state enabled<cr>
set ui_button myUI myBtnGroup state disabled<cr>
set ui_button myUI myBtnGroup state disabled 30<cr>
set ui_button myUI myBtnGroup state revert<cr>
set ui_button myUI myBtnGroup text up "myText"<cr>
set ui_button myUI myBtnGroup text down "line1\rline2"<cr>
set ui_button myUI myBtnGroup text both ""<cr>
set ui_button myUI myBtnGroup visibility true<cr>
set ui_button myUI myBtnGroup visibility false<cr>
set ui_button myUI mySplitBtnGroup position group (get matrix video)<cr>
set ui_button myUI mySplitDecoderBtn position down mySplitEncoderBtn<cr>
set ui_button myUI mySplitDecoderBtn position down null<cr>
set ui_button myUI mySplitDecoderBtn position clear<cr>
set ui_button key:abc123 myUI myBtn position up<cr>
```

3.5.7 Preset Examples

```
set ui_button myUI myBtn state disabled
set ui_button <<ui_name>> <<button_name>> position down
set ui_button <<ui_name>> (get var myElement) press
set ui_button <<ui_name>> myBtn image "myImg.png"
```

3.5.8 Response Examples

```
set ui_button position success<cr>
set ui_button state success<cr>
set ui_button text success<cr>
set ui_button clear success<cr>
set ui_button visibility success<cr>
set ui_button image success<cr>

set ui_button error [incomplete]<cr>
set ui_button error [ui '<ui_name>' not found]<cr>
set ui_button error [service disabled]<cr>
set ui_button error [button '<button_name>' not found]<cr>
set ui_button error [button disabled]<cr>
set ui_button error [invalid parameter]<cr>
set ui_button error [invalid function]<cr>
set ui_button error [invalid value '<value>']<cr>
set ui_button error [invalid button_position]<cr>
set ui_button error [invalid button_type]<cr>
set ui_button error [invalid split button]<cr>
set ui_button error [invalid time]<cr>
```

COMMAND GUIDE V1.3.8.0

3.6 Command set ui_label

3.6.1 Method

```
set ui_label [key:<security_key>] <ui_name> <label_name> <function> <value><cr>
```

3.6.2 Description

The command **set ui_label** is used to control a label within an active User Interface.

3.6.3 Arguments

argument	type	contains	
<ui_name>	string	Name of the User Interface	
<label_name>	string	Name of the label	
<function>	string	color bgcolor visibility text	
color	string	<value>	HEX RGB color code from 000000 to FFFFFF
bgcolor	string	<value>	HEX RGB color code from 000000 to FFFFFF false
visibility	boolean	<value>	true false
text	string	<value>	Object reference or a "wrapped string in quotes supporting line break \r"

3.6.4 Notes

- The UI service must be enabled.
- **bgcolor** <value> **false** applies a transparent background to the label.

3.6.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_label	<space>	success / error [message]	<cr>

3.6.6 Examples

```
set ui_label myUI myLbl text "myText"<cr>
set ui_label myUI myLbl text "line1\rline2"<cr>
set ui_label myUI myLbl text ""<cr>
set ui_label myUI myLbl text text_with_no_spaces<cr>
set ui_label myUI myLbl color 000000<cr>
set ui_label myUI myLbl bgcolor 000000<cr>
set ui_label myUI myLbl bgcolor false<cr>
set ui_label myUI myLbl visibility false<cr>
set ui_label myUI myLbl visibility true<cr>
set ui_label key:abc123 myUI myLbl text "myText"<cr>
```

COMMAND GUIDE V1.3.8.0

3.6.7 Preset Examples

```
set ui_label myUI myLbl text "myText"
set ui_label <<ui_name>> myLbl text "line1\rline2"
set ui_label <<ui_name>> (get var myElement) text ""
set ui_label <<ui_name>> myLbl text ("Selected Encoder: " & <<encoder_name>>)
set ui_label <<ui_name>> myLbl text ("Selected Decoder: " & <<decoder_name>>)
set ui_label <<ui_name>> myLbl text ("Selected Button: " & <<button_name>>)
set ui_label <<ui_name>> myLbl text ("Selected Button Position: " & <<button_position>>)
set ui_label <<ui_name>> myLbl text ("Selected Slider: " & <<slider_name>>)
set ui_label <<ui_name>> myLbl text ("Selected Slider Value: " & <<slider_value>>)
set ui_label <<ui_name>> myLbl text ("Entered code: " & <<pin_code>>)

set var x 60
loop while (x > 0) {
    set ui_label <<ui_name>> myLbl text ("Please Wait " & (get var x))
    preset delay 1000
    dec(x)
}
```

3.6.8 Response Examples

```
set ui_label color success<cr>
set ui_label bgcolor success<cr>
set ui_label visibility success<cr>
set ui_label text success<cr>

set ui_label error [incomplete]<cr>
set ui_label error [ui '<ui_name>' not found]<cr>
set ui_label error [label '<label_name>' not found]<cr>
set ui_label error [invalid parameter]<cr>
set ui_label error [invalid value '<value>']<cr>
set ui_label error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

3.7 Command set ui_image

3.7.1 Method

```
set ui_image [key:<security_key>] <ui_name> <image_name> <function> <value><cr>
```

3.7.2 Description

The command **set ui_image** is used to control an image within an active User Interface.

3.7.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<image_name>	string	Name of the image
<function>	string	visibility
<value>	boolean	true false

3.7.4 Notes

- The UI service must be enabled.
- Within a preset <<ui_name>> can replace <ui_name>.
- A visibility of false will also remove images from working layers allowing them to be placed over buttons.

3.7.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_image	<space>	<i>success / error [message]</i>	<cr>

3.7.6 Examples

```
set ui_image myUI myImg visibility false<cr>
set ui_image myUI myImg visibility true<cr>
set ui_image key:abc123 myUI myImg visibility false<cr>
```

3.7.7 Preset Examples

```
set ui_image myUI myImg visibility false
set ui_image <<ui_name>> myImg visibility false
set ui_image <<ui_name>> (get var myElement) visibility true
```

3.7.8 Response Examples

```
set ui_image visibility success<cr>

set ui_image error [incomplete]<cr>
set ui_image error [ui '<ui_name>' not found]<cr>
set ui_image error [image '<image_name>' not found]<cr>
set ui_image error [invalid parameter]<cr>
set ui_image error [invalid value '<value>']<cr>
set ui_image error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

3.8 Command set ui_page **updated*

3.8.1 Method

```
set ui_page [key:<security_key>] <ui_name> <page_name> [<delay>]<cr>
```

3.8.2 Description

The command **set ui_page** is used to change the displayed page in an active User Interface.

3.8.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<page_name>	string	Name of the page
<delay>	integer	Optional delay time in seconds from 0 to 180 or -1

3.8.4 Notes

- The UI service must be enabled.
- **home** is used as <page_name> to load the default page.
- **previous** is used as <page_name> to load the previous page.
- **login** is used as <page_name> to log out all users and load the login page.
- Within a preset <<ui_name>> can replace <ui_name>.
- <delay> has a range of **0** to **180** seconds before page change and **-1** will cancel the page change.
- Refer corresponding **get ui_page** command.

3.8.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_page	<space>	success / error [message]	<cr>

3.8.6 Examples

```
set ui_page myUI home<cr>
set ui_page myUI previous<cr>
set ui_page myUI login<cr>
set ui_page myUI myPage<cr>
set ui_page myUI myPage 60<cr>
set ui_page myUI myPage -1<cr>
set ui_page key:abc123 myUI home<cr>
```

3.8.7 Preset Examples

```
set ui_page myUI home
set ui_page <<ui_name>> myPage
set ui_page <<ui_name>> (get var myPage)
```

3.8.8 Response Examples

```
set ui_page success<cr>

set ui_page error [incomplete]<cr>
set ui_page error [ui '<ui_name>' not found]<cr>
set ui_page error [page '<page_name>' not found]<cr>
set ui_page error [invalid value '<value>']<cr>
set ui_page error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

3.9 Command set ui_indicator

3.9.1 Method

```
set ui_indicator [key:<security_key>] <ui_name> <indicator_name> <function> <value><cr>
```

3.9.2 Description

The command **set ui_indicator** is used to control a level indicator within an active User Interface.

3.9.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<indicator_name>	string	Name of the indicator
<function>	string	value
<value>	integer	-20000 ~ 20000 set by indicator settings min and max values

3.9.4 Notes

- The UI service must be enabled.
- A combined level slider will be update at the same time.
- Setting a <value> less than **min** will set the <value> as **min**.
- Setting a <value> greater than **max** will set the <value> as **max**.
- Obtain an amplifier volume status and set <value> with one process.
- Within a slider preset <<ui_name>> can replace <ui_name> and <<slider_value>> replace <value>.
- Refer corresponding **get ui_indicator** command.

3.9.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_indicator	<space>	success / error [message]	<cr>

3.9.6 Examples

```
set ui_indicator myUI myInd value 50<cr>
set ui_indicator key:abc123 myUI myInd value 50<cr>
```

3.9.7 Preset Examples

```
set ui_indicator myUI myInd value 50
set ui_indicator <<ui_name>> myInd value <<slider_value>>
set ui_indicator <<ui_name>> (get var myElement) value (get var sliderValue)
set ui_indicator myUI myInd value (val((send tcp 10.1.1.10 1234 "get volume\x0D" reply)))
```

3.9.8 Response Examples

```
set ui_indicator value success<cr>

set ui_indicator error [incomplete]<cr>
set ui_indicator error [ui '<ui_name>' not found]<cr>
set ui_indicator error [indicator '<indicator_name>' not found]<cr>
set ui_indicator error [invalid parameter]<cr>
set ui_indicator error [invalid value '<value>']<cr>
set ui_indicator error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

3.10 Command set ui_slider

3.10.1 Method

```
set ui_slider [key:<security_key>] <ui_name> <slider_name> <function> <value><cr>
```

3.10.2 Description

The command **set ui_slider** is used to control a level slider within an active User Interface.

3.10.3 Arguments

argument	type	contains	
<ui_name>	string	Name of the User Interface	
<slider_name>	string	Name of the slider	
<function>	string	value state	
value	integer	<value>	-20000 ~ 20000 set by slider settings min and max values
state	string	<value>	enabled disabled

3.10.4 Notes

- The UI service must be enabled.
- A combined level indicator will be update at the same time.
- Setting a <value> less than **min** will set the <value> as **min**.
- Setting a <value> greater than **max** will set the <value> as **max**.
- Within a preset <<ui_name>> can replace <ui_name>.
- Obtain an amplifier volume status and set <value> with one process.
- Within a slider preset <<slider_name>> can replace <slider_name> and <<slider_value>> replace <value>.
- Refer corresponding **get ui_slider** command.

3.10.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_slider	<space>	success / error [message]	<cr>

3.10.6 Examples

```
set ui_slider myUI mySlidr value 50<cr>
set ui_slider myUI mySlidr state disabled<cr>
set ui_slider myUI mySlidr state enabled<cr>
set ui_slider key:abc123 myUI mySlidr value 50<cr>
```

3.10.7 Preset Examples

```
set ui_slider myUI mySlidr value 50
set ui_slider <<ui_name>> <<slider_name>> value <<slider_value>>
set ui_slider <<ui_name>> (get var myElement) value (get var sliderValue)
set ui_slider myUI mySlidr value (val((send tcp 10.1.1.10 1234 "get volume\x0D" reply)))
```

3.10.8 Response Examples

```
set ui_slider value success<cr>
set ui_slider state success<cr>
```

```
set ui_slider error [incomplete]<cr>
set ui_slider error [ui '<ui_name>' not found]<cr>
set ui_slider error [slider '<slider_name>' not found]<cr>
set ui_slider error [invalid value '<value>']<cr>
set ui_slider error [invalid parameter]<cr>
set ui_slider error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

3.11 Command set ui_redirect

3.11.1 Method

```
set ui_redirect [key:<security_key>] <current_ui_name> <redirected_ui_name> [<page_name>]<cr>
```

3.11.2 Description

The command **set ui_redirect** is used to change a clients current User Interface from one to another. Ideal for combining rooms where two separate User Interfaces are used, but once the rooms are combined a common User Interface is required.

3.11.3 Arguments

argument	type	contains
<current_ui_name>	string	Name of the current User Interface
<redirected_ui_name>	string	Name of the replacement User Interface
<page_name>	string	Optional page selection of the replacement User Interface

3.11.4 Notes

- The UI service must be enabled.
- The redirected UI must be reverted with command **set ui_revert** before using **set ui_redirect** on it again.
- Refer corresponding **set ui_revert** command.

3.11.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_redirect	<space>	success / error [message]	<cr>

3.11.6 Examples

```
set ui_redirect myUI myNewUI<cr>
set ui_redirect myUI myNewUI page1<cr>
set ui_redirect key:abc123 myUI myNewUI<cr>
```

3.11.7 Preset Examples

```
set ui_redirect myUI myNewUI
set ui_redirect <<ui_name>> myNewUI

set var origianlUI <<ui_name>>
set ui_redirect <<ui_name>> (get var myNewUI)
```

3.11.8 Response Examples

```
set ui_redirect success<cr>

set ui_redirect error [incomplete]<cr>
set ui_redirect error [duplicate]<cr>
set ui_redirect error [ui '<ui_name>' not found]<cr>
set ui_redirect error [page '<page_name>' not found]<cr>
set ui_redirect error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

3.12 Command set ui_revert

3.12.1 Method

```
set ui_revert [key:<security_key>] <original_ui_name><cr>
```

3.12.2 Description

The command **set ui_revert** is used to revert a clients User Interface to the original, before a **ui_redirect**.

3.12.3 Arguments

argument	type	contains
<original_ui_name>	string	Name of the original redirected User Interface

3.12.4 Notes

- The UI service must be enabled.
- Refer corresponding **set ui_redirect** command.

3.12.5 Response

command	<space>	mode	<space>	status	terminator
set	<space>	ui_revert	<space>	success / error [message]	<cr>

3.12.6 Examples

```
set ui_revert myUI<cr>
set ui_revert key:abc123 myUI<cr>
```

3.12.7 Preset Examples

```
set ui_revert myUI
set ui_revert (get var origianlUI)
```

3.12.8 Response Examples

```
set ui_revert success<cr>

set ui_revert error [incomplete]<cr>
set ui_revert error [ui '<original_ui_name>' not found]<cr>
set ui_revert error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

4 Command get

The **get** commands are used to retrieve various information.

4.1 Command get api

4.1.1 Method

```
get api [key:<security_key>]<cr>
```

4.1.2 Description

The command **get api** is used to retrieve the API version.

4.1.3 Arguments

None

4.1.4 Notes

- Use the command **get api** in a preset to determine what commands can be used.

4.1.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	api	<space>	success data / error [message]	<space>	<string>	<cr>

4.1.6 Command Example

```
get api<cr>
get api key:abc123<cr>
```

4.1.7 Return Example

```
get api success 1.3.8.0<cr>
```

COMMAND GUIDE V1.3.8.0

4.2 Command get events

4.2.1 Method

```
get events [key:<security_key>] <event_name> <function><cr>
```

4.2.2 Description

The command **get events** is used to retrieve the state of events created from the Scheduler tab.

4.2.3 Arguments

argument	type	contains
<event_name>	string	Name of the event
<function>	string	state

4.2.4 Notes

- Event must be created on the UI Scheduler tab before using command.
- Refer corresponding **set events** command.

4.2.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	events	<space>	success data / error [message]	<space>	<string>	<cr>

4.2.6 Example

```
get events state myEvent<cr>
get events key:abc123 state myEvent<cr>
```

4.2.7 Response Examples

```
get events success enabled<cr>
get events success disabled<cr>

get events error [incomplete]<cr>
get events error [event '<event_name>' not found]<cr>
get events error [invalid parameter]<cr>
```

COMMAND GUIDE V1.3.8.0

4.3 Command get var

4.3.1 Method

```
get var [key:<security_key>] <var_name><cr>
```

4.3.2 Description

The command **get var** is used to retrieve the value of the specified user defined variable.

4.3.3 Arguments

argument	type	contains
<var_name>	string	Name of the variable

4.3.4 Notes

- Refer **Appendix A** – Preset Logic.
- Refer corresponding **set var** command.

4.3.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	var	<space>	success data / error [message]	<space>	<string>	<cr>

4.3.6 Examples

```
get var myVar<cr>
get var key:abc123 myVar<cr>
```

4.3.7 Preset Examples

```
if (get var myVar) {
    //variable exists or boolean true
} else {
    //variable does not exist or boolean false
}

set var myElement "myBtn"
set ui_button <<ui_name>> (get var myElement) state disabled
```

4.3.8 Response Examples

```
get var success <value><cr>

get var error [incomplete]<cr>
get var error [var '<var_name>' not found]<cr>
```

COMMAND GUIDE V1.3.8.0

4.4 Command get ui

4.4.1 Method

```
get ui [key:<security_key>] <ui_name><cr>
```

4.4.2 Description

The command **get ui** is used to retrieve a User Interface status.

4.4.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface

4.4.4 Notes

- The UI service must be enabled.
- Within a preset <<ui_name>> can replace <ui_name>.
- Refer corresponding **set ui** command.

4.4.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	ui	<space>	success data / error [message]	<space>	<string>	<cr>

4.4.6 Example

```
get ui myUI<cr>
get ui key:abc123 myUI<cr>
```

4.4.7 Preset Example

```
if ((get ui myUI) == "disabled") {
    set ui myUI enabled
}
```

4.4.8 Response Examples

```
get ui success disabled<cr>
get ui success enabled<cr>
get ui success enabled clients 1<cr> *number of users 1 to 100
get ui success enabled login 1234<cr> *0000 to 9999
get ui success enabled clients 1 login 1234<cr>
get ui success enabled timeout 240 clients 1 login 1234<cr>

get ui error [incomplete]<cr>
get ui error [ui '<ui_name>' not found]<cr>
get ui error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

4.5 Command get ui_button

**updated (future functionality pending)*

4.5.1 Method

`get ui_button [key:<security_key>] <ui_name> <button_name> <function> [<button_position>]<cr>`

4.5.2 Description

The command **get ui_button** is used to retrieve the current condition of a User Interface button.

4.5.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<button_name>	string	Name of the button
<function>	string	position state text visibility image

4.5.4 Notes

- The UI service must be enabled.
- Use the command **set ui_button** to change this setting.
- **<function> position** returns **up** | **down** | **hold**
- **<function> state** returns **enabled** | **disabled**
- **<function> visibility** uses **true** | **false**
- **<function> image** along with **position up** | **down** returns the image name as a string.
- **<button_position>** used with **<function> text** for position **up** | **down**
- Refer corresponding **set ui_button** command.
- **Momentary**
 - **position**
 - state
 - text (up)
 - visibility
 - image (up)
- **Toggle**
 - position
 - state
 - text
 - visibility
 - image
- **Radio Toggle**
 - position
 - state
 - text
 - visibility
 - image
- **Repeat**
 - **position**
 - state
 - text (up)
 - visibility
 - image (up)
- **Split**
 - position
 - state
 - text
 - visibility
 - image
- **QR Code**
 - state
 - visibility

4.5.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	ui_button	<space>	success data / error [message]	<space>	<string>	<cr>

4.5.6 Examples

```
get ui_button myUI myBtn position<cr>
get ui_button myUI myBtn state<cr>
get ui_button myUI myBtn text up<cr>
get ui_button myUI myBtn text down<cr>
get ui_button myUI myBtn visibility<cr>
get ui_button myUI myBtn image up<cr>
get ui_button myUI myBtn image down<cr>
get ui_button key:abc123 myUI myBtn position<cr>
```

COMMAND GUIDE V1.3.8.0

4.5.7 Preset Examples

```
if (get ui_button <<ui_name>> <<button_name>> position == "down") {  
    send tcp 10.1.1.10 1234 "myCmd"  
}  
  
if (get ui_button <<ui_name>> (get var myElement) state == "disabled") {  
    set ui_button <<ui_name>> (get var myElement) state enabled  
}  
  
set var myBtnImg (get ui_button myUI myBtn image up)  
set ui_button myUI myBtn image up "error.png"  
preset delay 3000  
set ui_button myUI myBtn image up (get var myBtnImg)
```

4.5.8 Response Examples

```
get ui_button position success up<cr>  
get ui_button position success down<cr>  
get ui_button state success enabled<cr>  
get ui_button state success disabled<cr>  
get ui_button text success <text><cr>  
get ui_button visibility success true<cr>  
get ui_button visibility success false<cr>  
get ui_button image success myImg.png<cr>  
  
get ui_button error [incomplete]<cr>  
get ui_button error [ui '<ui_name>' not found]<cr>  
get ui_button error [button '<button_name>' not found]<cr>  
get ui_button error [invalid function]<cr>  
get ui_button error [invalid button type]<cr>  
get ui_button error [invalid button_position]<cr>  
get ui_button error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

4.6 Command get ui_indicator

4.6.1 Method

```
get ui_indicator [key:<security_key>] <ui_name> <indicator_name> <function><cr>
```

4.6.2 Description

The command **get ui_indicator** is used to retrieve the current condition of a User Interface level indicator.

4.6.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<indicator_name>	string	Name of the level indicator
<function>	string	value

4.6.4 Notes

- The UI service must be enabled.
- Within a preset <<ui_name>> can replace <ui_name>.
- Refer corresponding **set ui_indicator** command.

4.6.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	ui_indicator	<space>	success data / error [message]	<space>	<string>	<cr>

4.6.6 Examples

```
get ui_indicator myUI myInd value<cr>
get ui_indicator key:abc123 myUI myInd value<cr>
```

4.6.7 Preset Examples

```
set var myVar (get ui_indicator myUI myInd value)
set var myVar (get ui_indicator <<ui_name>> (get var myElement) value)
set ui_slider <<ui_name>> mySldr value (get ui_indicator <<ui_name>> myInd value)
```

4.6.8 Response Examples

```
get ui_indicator value success 50<cr>

get ui_indicator error [incomplete]<cr>
get ui_indicator error [ui '<ui_name>' not found]<cr>
get ui_indicator error [indicator '<indicator_name>' not found]<cr>
get ui_indicator error [invalid parameter]<cr>
get ui_indicator error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

4.7 Command get ui_slider

4.7.1 Method

```
get ui_slider [key:<security_key>] <ui_name> <slider_name> <function><cr>
```

4.7.2 Description

The command **get ui_slider** is used to retrieve the current condition of a User Interface level slider.

4.7.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface
<slider_name>	string	Name of the level slider
<function>	string	value state

4.7.4 Notes

- The UI service must be enabled.
- **<function> value** returns integer.
- **<function> state** returns **enabled | disabled**.
- Within a preset <<ui_name>> can replace <ui_name>.
- Within a slider preset <<slider_name>> can replace <slider_name>.
- Refer corresponding **set ui_slider** command.

4.7.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	ui_slider	<space>	success data / error [message]	<space>	<string>	<cr>

4.7.6 Examples

```
get ui_slider myUI mySlidr value<cr>
get ui_slider myUI mySlidr state<cr>
get ui_slider key:abc123 myUI mySlidr value<cr>
```

4.7.7 Preset Examples

```
set var myVar (get ui_slider myUI mySlidr value)
set var myVar (get ui_slider <<ui_name>> (get var myElement) value)
set ui_indicator <<ui_name>> myInd value (get ui_slider <<ui_name>> mySlidr value)
```

4.7.8 Response Examples

```
get ui_slider value success 50<cr>
get ui_slider state success enabled<cr>
get ui_slider state success disabled<cr>

get ui_slider error [incomplete]<cr>
get ui_slider error [ui '<ui_name>' not found]<cr>
get ui_slider error [slider '<slider_name>' not found]<cr>
get ui_slider error [invalid parameter]<cr>
get ui_slider error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

4.8 Command get ui_page (new)

4.8.1 Method

```
get ui_page [key:<security_key>] <ui_name><cr>
```

4.8.2 Description

The command **get ui_page** is used to retrieve the current page name displayed on the specified UI.

4.8.3 Arguments

argument	type	contains
<ui_name>	string	Name of the User Interface

4.8.4 Notes

- The UI service must be enabled.
- The UI's current page name will be returned and not any clients login page.
- Within a preset <<ui_name>> can replace <ui_name>.
- Refer corresponding **set ui_page** command.

4.8.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
get	<space>	ui_page	<space>	success data / error [message]	<space>	<string>	<cr>

4.8.6 Examples

```
get ui_page myUI<cr>
get ui_page key:abc123 myUI<cr>
```

4.8.7 Preset Examples

```
set var myVar (get ui_page myUI)
set var myVar (get ui_page <<ui_name>>)

if (get ui_page <<ui_name>> != "myPage") {
    set ui_label <<ui_name>> myLbl text (get ui_page <<ui_name>>)
}
```

4.8.8 Response Examples

```
get ui_page success home<cr>
get ui_page success YourPageName<cr>

get ui_page error [incomplete]<cr>
get ui_page error [ui '<ui_name>' not found]<cr>
get ui_page error [service disabled]<cr>
```

COMMAND GUIDE V1.3.8.0

5 Command send

The **send** commands are used to send either **TCP**, **Telnet**, **UDP** or **Global Caché** commands **including email notifications**. Any TCP, Telnet, UDP and Global Caché device can be controlled.

References in this manual to \x0D and <cr> both refer to a carriage return or \r, hex byte 0x0D, decimal 13. References in this manual to \x0A and <lf> both refer to a linefeed or \n, hex byte 0x0A, decimal 10.

The use of \xYY is the standard method of using and displaying all hexadecimal bytes with ASCII. The use of \xYY within any ASCII command string will be converted into a hexadecimal byte before sending.

Any received hexadecimal bytes will be escaped the same with \xYY in ASCII mode, so often the peripheral's response string is terminated with \x0D or \x0D\x0A. This is something to be aware of when using <feedback> comparison option **equals** over **contains**. The specified **equals** string must also include all terminators or otherwise hidden hexadecimal non ASCII characters. The **trim()** function can be used to remove any trailing non ASCII characters.

Sending any hexadecimal byte value \x00 to \xFF in ASCII mode is supported.

Only non standard ASCII hexadecimal characters received will be terminated with \x in ASCII mode.

Optional HEX mode will allow sending and receiving hexadecimal bytes without the need to escape with \x.

<cr> is used to specifically show the termination of the Arranger API command.

Using feedback options **contains** or **exact** will usually return as success/true or error/false as a result of the feedback comparison. If multiple response options or a value need to be processed then usually a feedback **reply** would be used and logic applied to process the action of the response string.

With the use of **action** and **actionelse** this logic can be applied at the command line level instead.

```
send tcp 10.1.1.10 1234 "sourceStatus" ...
  contains "source01" action (set ui_button "myUI" "myBtn01" position down) ...
  contains "source02" action (set ui_button "myUI" "myBtn02" position down) ...
  contains "source03" action (set ui_button "myUI" "myBtn03" position down) ...
  actionelse (set ui_button "myUI" "myBtnGroup" position up)<cr>
```

The above command line is equivalent to the below preset logic:

```
set var myResponse (send tcp 10.1.1.10 1234 "sourceStatus" reply)

if ((instr((get var myResponse),"source01")) {
  set ui_button "myUI" "myBtn01" position down
} elseif ((instr((get var myResponse),"source02")) {
  set ui_button "myUI" "myBtn02" position down
} elseif ((instr((get var myResponse),"source03")) {
  set ui_button "myUI" "myBtn03" position down
} else {
  set ui_button "myUI" "myBtnGroup" position up
}
```

5 Command send continued...

```
send tcp 10.1.1.10 1234 "getVolume" ...
  contains "volume" action (set ui_slider "myUI" "mySldr" value (val(<<send_response>>,[<offset>])))
```

The above command line is equivalent to the below preset logic:

```
set var myResponse (send tcp 10.1.1.10 1234 "getVolume" reply)
set ui_slider "myUI" "mySldr" value (val((get var myResponse),[<offset>]))
```

```
send tcp 10.1.1.10 1234 "sourceStatus" ...
  exact "source01\x0D" action (set ui_button "myUI" "myBtn01" position down) ...
  exact "source02\x0D" action (set ui_button "myUI" "myBtn02" position down) ...
  exact "source03\x0D" action (set ui_button "myUI" "myBtn03" position down) ...
  actionelse (set ui_button "myUI" "myBtnGroup" position up)<cr>
```

The above command line is equivalent to the below preset logic:

```
set var myResponse (send tcp 10.1.1.10 1234 "getVolume" reply)

select case ((get var myResponse)) {
  case "source01\x0D"
    set ui_button "myUI" "myBtn01" position down
  case "source02\x0D"
    set ui_button "myUI" "myBtn02" position down
  case "source03\x0D"
    set ui_button "myUI" "myBtn03" position down
  case else
    set ui_button "myUI" "myBtnGroup" position up
}
```

COMMAND GUIDE V1.3.8.0

5 Command send continued...**send tcp**

The command **send tcp** will open a TCP connection to the specified IP Address and Port then send the specified command string. The TCP connection will be maintained for 45 seconds for any consecutive commands before automatically disconnected.

The feedback operation is optional as follows:

- **No feedback required**

When no command response is required, then no **<feedback>** parameters are specified.

The send tcp command result is success or error, Boolean true or false, as a result of the connection.

Examples:

```
send tcp 10.1.1.10 1234 "my command"<cr>
send tcp 10.1.1.10 1234 "6D7920636F6D6D616E64" hex<cr>
```

- **Command feedback**

When command response is required, the **<feedback>** keyword **reply** is used.

A successful send tcp command result is the returned string only.

Examples:

```
send tcp 10.1.1.10 1234 "my command" reply<cr>
send tcp 10.1.1.10 1234 "6D7920636F6D6D616E64" hex reply<cr>
```

- **Comparison contains**

When a partial response match is required the **<feedback>** keyword **contains** along with the string to find are specified.

The send tcp command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Examples:

```
send tcp 10.1.1.10 1234 "my command\x0D" contains "my response"<cr>
send tcp 10.1.1.10 1234 "6D7920636F6D6D616E640D" hex contains "6D7920726573706F6E7365"<cr>
```

- **Comparison equals**

When an exact response match is required the **<feedback>** keyword **equals** along with the string to find are specified.

The send tcp command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Examples:

```
send tcp 10.1.1.10 1234 "my command\x0D" equals "my response\x0D"<cr>
send tcp 10.1.1.10 1234 "6D7920636F6D6D616E640D" hex equals "6D7920726573706F6E73650D"<cr>
```

5 Command send continued...

send gc

The command send gc is similar to send tcp except operates with specific Global Caché conditions. It will also open a TCP connection to the specified IP Address and Port then send the specified Global Caché command string automatically appended with <cr> when using port 4998. The TCP connection will be maintained for 45 seconds for any consecutive commands before being disconnected.

Variations of send tcp include the following:

- **Reserved port numbers**

Only Global Caché ports 4998, 4999 & 5000 are valid.

- **Command string**

All command strings are strictly Global Caché API commands without the need for termination with \x0D within the command string itself. This will automatically be appended before sending.

The send gc command itself still needs to be terminated with <cr>.

Example:

```
send gc 169.254.1.70 4998 "setstate,1:1,1"<cr>
```

- **Command results**

Command result depends on the network port used.

Commands using port 4998 result in Boolean true | false relative to the specific function specified.

So for example a relay closure command "setstate,1:1,1" will confirm the correct device response.

Example:

```
send gc 169.254.1.70 4998 "<globalcaché_command>"<cr>
```

5 Command send continued...

Commands using serial port 4999 or 5000 can use optional feedback.

- **No feedback required**

When no command response is required, then no **<feedback>** parameters are specified.

The send gc command result is success or error, Boolean true or false, as a result of the connection.

Examples:

```
send gc 169.254.1.70 4999 "<string>"<cr>
send gc 169.254.1.70 5000 "<string>"<cr>
```

- **Command feedback**

When command response is required, the **<feedback>** keyword **reply** is used.

The send gc command result is the returned string only.

Examples:

```
send gc 169.254.1.70 4999 "<string>" reply<cr>
send gc 169.254.1.70 5000 "<string>" reply<cr>
```

- **Comparison contains**

When a partial response match is required the **<feedback>** keyword **contains** along with the string to find are specified.

The send gc command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Examples:

```
send gc 169.254.1.70 4999 "<string>" contains "<response>"<cr>
send gc 169.254.1.70 5000 "<string>" contains "<response>"<cr>
```

- **Comparison equals**

When an exact response match is required the **<feedback>** keyword **equals** along with the string to find are specified.

The send gc command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Examples:

```
send gc 169.254.1.70 4999 "<string>" equals "<response>\x0D"<cr>
send gc 169.254.1.70 5000 "<string>" equals "<response>\x0D"<cr>
```

COMMAND GUIDE V1.3.8.0

5 Command send continued...

send udp

The command send udp will open a UDP connection to the specified IP Address and Ports then send the specified command string. The UDP connection will be maintained for 10 seconds for any consecutive commands before being dropped.

The feedback operation is optional as follows:

- **No feedback required**

When no command response is required, then no **<feedback>** parameters are specified.

The send udp command result is success or error, Boolean true or false, as a result of the connection.

Examples:

```
send udp 255.255.255.255 1234 "my command"<cr>
send udp 255.255.255.255 1234 "6D7920636F6D6D616E64" hex<cr>
```

- **Command feedback**

When command response is required, the **<feedback>** keyword **reply** is used.

The send udp command result is the returned string only.

Examples:

```
send udp 255.255.255.255 1234 5678 "my command" reply<cr>
send udp 255.255.255.255 1234 5678 "6D7920636F6D6D616E64" hex reply<cr>
```

- **Comparison contains**

When a partial response match is required the **<feedback>** keyword **contains** along with the string to find are specified.

The send udp command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Examples:

```
send udp 255.255.255.255 1234 5678 "my command\x0D" contains "my response"<cr>
send udp 255.255.255.255 1234 5678 "6D7920636F6D6D616E640D" hex contains "6D7920726573706F6E7365"<cr>
```

- **Comparison equals**

When an exact response match is required the **<feedback>** keyword **equals** along with the string to find are specified.

The send udp command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Examples:

```
send udp 255.255.255.255 1234 5678 "my command\x0D" equals "my response\x0D"<cr>
send udp 255.255.255.255 1234 5678 "6D7920636F6D6D616E640D" hex equals "6D7920726573706F6E73650D"<cr>
```

COMMAND GUIDE V1.3.8.0

5 Command send continued...**send telnet**

The command send telnet will open a TCP connection to the specified IP Address and Port then send the specified command string along with login details. The TCP connection will be maintained until disconnected.

The feedback operation is optional as follows:

- **No feedback required**

When no response is required, then no **<feedback>** parameters are specified.
The send telnet command result is success or error, Boolean true or false, as a result of the connection.

Example:

```
send telnet 10.1.1.10 23 username "User:" "MyLogin" "my command"<cr>
```

- **External device feedback**

When a command response is required, the **<feedback>** keyword **reply** is used.
The send telnet command result is the returned string only.

Example:

```
send telnet 10.1.1.10 23 username "User:" "MyLogin" "my command" reply<cr>
```

- **Comparison contains**

When a partial response match is required the **<feedback>** keyword **contains** along with the string to find are specified.
The send telnet command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Example:

```
send telnet 10.1.1.10 23 username "User:" "MyLogin" "my command\x0D" contains "my response"<cr>
```

- **Comparison equals**

When an exact response match is required the **<feedback>** keyword **equals** along with the string to find are specified.
The send telnet command result is success or error, Boolean true or false, as a result of the connection and string comparison.

Example:

```
send telnet 10.1.1.10 23 username "User:" "MyLogin" "my command\x0D" equals "my response\x0D"<cr>
```

COMMAND GUIDE V1.3.8.0

5.1 Command send gc (future functionality pending)

The command **send gc** provides a seamless integration with Global Caché products. For more information on Global Caché visit www.globalcache.com.

5.1.1 Method

```
send gc [key:<security_key>] <address> <port> <gc_api> ["<feedback_string>" [<action> (<process>)]]<cr>
```

5.1.2 Description

The **send gc** command allows control of all Global Caché products via TCP.

5.1.3 Arguments

argument	type	contains	
<address>	string	Global Caché IP address	
<port>	integer	Global Caché TCP port. Usually 4998 and for serial COM1: 4999 COM2: 5000	
<gc_api>	string	Global Caché API command	
<feedback>	string	Optional feedback keyword reply equals contains	
reply	string	Return with response string	
equals	string	<feedback_string>	Complete response string
contains	string	<feedback_string>	Partial response string
<action>	string	Optional equals or contains action keyword action actionelse	
(<process>)	string	Command line wrapped with brackets	

5.1.4 Notes

- The Global Caché device must be accessible by the controller.
- The **<gc_api>** string uses the same standard Global Caché control string format as found in the Global Caché API manuals, downloaded from: www.globalcache.com/downloads/.
- Use keyword [**disconnect**] in place of **<gc_api>** to terminate the connection.
- A TCP connection will be maintained for 45 seconds before automatically disconnecting.
- Serial strings sent to port **4999** or **5000** must be wrapped in quotation marks as "*my string*".
- To send serial hexadecimal bytes add \x before each byte. \x0D for carriage return.
- The **<feedback>** option uses keywords **reply**, **equals** or **contains** to set the type of feedback required. To receive a response only, use **reply**. For comparison with the specified **<feedback_string>** use **equals** for an exact match, or **contains** for a match within the string.
- Use multiple feedback **contains** | **exact** options with **action** for status updates to perform UI functions as a result of the response received.
- **action** is a success response while **actionelse** is an error response.
- System variable **<<send_response>>** will be set with the response string.

COMMAND GUIDE V1.3.8.0

5.1.5 Response

Responses from port 4998 will be in the standard Global Caché format under normal conditions.

An exception to this would be if a TCP connection was not possible to a device, in which case an error such as `send gc error [device '169.254.1.70' not found]<cr>` would be sent.

- | |
|---|
| success ● establishing a TCP connection
● command result successful
error ● not able to establish a TCP connection
● command result not successful |
|---|

Responses from serial ports 4999 or 5000 will be the returned serial string for keyword **reply**, keyword **contains** or **equals** will return with a success or error as a result of the connection and comparison of the serial returned string.

<feedback> null	success ● establishing a TCP connection error ● not able to establish a TCP connection
<feedback> reply	success ● establishing a TCP connection ○ received string included in the send gc command success response error ● not able to establish a TCP connection
<feedback> contains	success ● establishing a TCP connection ● <feedback_string> found in the received string error ● not able to establish a TCP connection ● <feedback_string> not found in the received string ○ received string included in the send gc command error response
<feedback> exact	success ● establishing a TCP connection ● <feedback_string> matches the received string error ● not able to establish a TCP connection ● <feedback_string> not matching the received string ○ received string included in the send gc command error response

5.1.6 Examples

```
send gc 169.254.1.70 4999 "an ascii string"<cr>
send gc 169.254.1.70 4999 "a mixed string\x0D"<cr>
send gc 169.254.1.70 4999 "\x61\x20\x48\x45\x58\x20\x73\x74\x72\x69\x6E\x67"<cr>
send gc 169.254.1.70 4999 "command string" reply<cr>
send gc 169.254.1.70 4999 "command string" contains "response string"<cr>
send gc 169.254.1.70 4999 "command string" equals "response string\x0D\x0A"<cr>
send gc 169.254.1.70 4998 setstate,1:1,1<cr>
send gc 169.254.1.70 4998 getstate,1:1<cr>
send gc 169.254.1.70 4998 getstate,1:1 contains 1:1,1<cr>
send gc 169.254.1.70 4998 sendir,1:1,4444,34500,1,1,34,48,24,12,24,960,24,12,24...<cr>
send gc 169.254.1.70 4998 [disconnect]<cr>
send gc key:abc123 169.254.1.70 4999 "an ascii string"<cr>
```

COMMAND GUIDE V1.3.8.0

5.1.6 Examples continued...

```
send gc 169.254.1.70 4999 "sourceStatus" ...
  contains "source01" action (set ui_button "myUI" "myBtn01" position down) ...
  contains "source02" action (set ui_button "myUI" "myBtn02" position down) ...
  contains "source03" action (set ui_button "myUI" "myBtn03" position down) ...
  actionelse (set ui_button "myUI" "myBtnGroup" position up)<cr>

send gc 169.254.1.70 4999 "sourceStatus" contains "source01" ...
  actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send gc 169.254.1.70 4999 "sourceStatus" exact "source01" ...
  action (set ui_button "myUI" "myBtn01" position down) ...
  actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send gc 169.254.1.70 4999 "sourceStatus" exact "source01" ...
  actionelse (set ui_button "myUI" "myBtn01" position up)<cr>
```

5.1.7 Preset Examples

```
send gc 169.254.1.70 4999 "a string to send"
send gc 169.254.1.70 4998 setstate,1:1,1
send gc (get var myIP) 4998 sendir,1:2,4444,34500,1,1,34,48,24,12,24,960,24,12,24...

if (send gc 169.254.1.70 4998 setstate,1:1,1) {
  //success
} else {
  //error
}

if (send gc 169.254.1.70 4998 getstate,1:1 contains 1:1,1) {
  //on
} else {
  //off
}

send gc 169.254.1.70 4999 "sourceStatus" ...
  contains "source01" action (set ui_button <<ui_name>> "myBtn01" position down) ...
  contains "source02" action (set ui_button <<ui_name>> "myBtn02" position down) ...
  contains "source03" action (set ui_button <<ui_name>> "myBtn03" position down) ...
  actionelse (set ui_button <<ui_name>> "myBtnGroup" position up)

send gc 169.254.1.70 4999 "state" exact "s01" actionelse (set ui_button "myUI" "myBtn01" position up)

send gc 169.254.1.70 4999 "command" reply
set ui_label <<ui_name>> "myLb1" text <<send_response>>

send gc 169.254.1.70 4999 "getVolume" reply
set ui_slider <<ui_name>> "mySlidr" value (val(<<send_response>>))
```

5.1.8 Response Examples

```
send gc success<cr>
a serial string<cr>
state,1:1,1<cr>
completeir,1:1,1<cr>

send gc error [invalid]<cr>
send gc error [invalid format]<cr>
send gc error [invalid port]<cr>
send gc error [incomplete]<cr>
send gc error [device '<address>' not found]<cr>
send gc error [device '<address>' timeout]<cr>
send gc error [<string received>]<cr>
send error [invalid mode]<cr>
ERR_<XX><cr>
```

COMMAND GUIDE V1.3.8.0

5.2 Command send tcp (future functionality pending)

5.2.1 Method

```
send tcp [key:<security_key>] <address> <port> "<command>" [<protocol>] [<feedback> ["<feedback_string>" [<action> (<process>)]]]<cr>
```

5.2.2 Description

The command **send tcp** provides a seamless integration with any TCP controllable device.

5.2.3 Arguments

argument	type	contains	
<address>	string	TCP IP address	
<port>	integer	TCP port	
<command>	string	Command string to send to TCP device	
<protocol>	string	Optional keyword ascii (default) hex	
<feedback>	string	Optional feedback keyword reply equals contains	
reply	string	Return with response string	
equals	string	<feedback_string>	Complete response string
contains	string	<feedback_string>	Partial response string
<action>	string	Optional equals or contains action keyword action actionelse	
(<process>)	string	Command line wrapped with brackets	

5.2.4 Notes

- The TCP device must be accessible by the controller.
- To send hexadecimal bytes with protocol ASCII add \x before each byte. \x0D for carriage return.
- Optional keyword **hex** allows hexadecimal bytes to be sent and received without the need for \x.
- The <feedback> option uses keywords **reply**, **equals** or **contains** to set the type of feedback required. To receive a string only, use **reply**. For comparison with the specified <feedback_string> use **equals** for an exact match, or **contains** for a match within the string.
- <feedback_string> contains the expected response string.
- Use double quotes "" within a "wrapped" string to send a quotation mark ".
send tcp <address> <port> "wrap this value ""10""\x0D"<cr> will send *wrap this value "10"<cr>*
Blue are command wrappers for "<string>", Green are trigger "wrappers" with Black wrappers within "<string>".
- Use keyword **[disconnect]** in place of <command> to terminate the TCP connection.
- The TCP connection will be maintained for 45 seconds before automatically disconnecting.
- Use multiple feedback **contains** | **exact** options with **action** for status updates to perform UI functions as a result of the response received.
- action** is a success response while **actionelse** is an error response.
- System variable <<send_response>> will be set with the response string.

COMMAND GUIDE V1.3.8.0

5.2.5 Response

<feedback> null	success ● establishing a TCP connection error ● not able to establish a TCP connection
<feedback> reply	success ● establishing a TCP connection ○ received string included in the send tcp command success response error ● not able to establish a TCP connection
<feedback> contains	success ● establishing a TCP connection ● <feedback_string> found in the received string error ● not able to establish a TCP connection ● <feedback_string> not found in the received string ○ received string included in the send tcp command error response
<feedback> exact	success ● establishing a TCP connection ● <feedback_string> matches the received string error ● not able to establish a TCP connection ● <feedback_string> not matching the received string, ○ received string included in the send tcp command error response

5.2.6 Examples

```
send tcp 10.1.1.10 1234 "an ascii string"<cr>
send tcp 10.1.1.10 1234 "a mixed string\x0D"<cr>
send tcp 10.1.1.10 1234 "\x61\x20\x48\x45\x58\x20\x73\x74\x72\x69\x6E\x67"<cr>
send tcp 10.1.1.10 1234 "command string" contains "response string"<cr>
send tcp 10.1.1.10 1234 "command string" equals "response string\x0D\x0A"<cr>
send tcp 10.1.1.10 1234 "command string" reply<cr>
send tcp 10.1.1.10 1234 "6D792068657820636F6D6D616E64" hex reply<cr>
send tcp 10.1.1.10 1234 "6D792068657820636F6D6D616E64" hex equals "6D792068657820666565646261636B"<cr>
send tcp 10.1.1.10 1234 "set vol ""10""\x0D"<cr>
send tcp 10.1.1.10 1234 [disconnect]<cr>
send tcp key:abc123 10.1.1.10 1234 "an ascii string"<cr>
```

```
send tcp 10.1.1.10 1234 "sourceStatus" ...
contains "source01" action (set ui_button "myUI" "myBtn01" position down) ...
contains "source02" action (set ui_button "myUI" "myBtn02" position down) ...
contains "source03" action (set ui_button "myUI" "myBtn03" position down) ...
actionelse (set ui_button "myUI" "myBtnGroup" position up)<cr>
```

```
send tcp 10.1.1.10 1234 "sourceStatus" contains "source01" ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>
```

```
send tcp 10.1.1.10 1234 "sourceStatus" exact "source01" ...
action (set ui_button "myUI" "myBtn01" position down) ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>
```

```
send tcp 10.1.1.10 1234 "sourceStatus" exact "source01" ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>
```

COMMAND GUIDE V1.3.8.0

5.2.7 Preset Examples

```
send tcp 10.1.1.10 1234 "myCommand"
send tcp 10.1.1.10 1234 "6D792068657820636F6D6D616E64" hex reply

if (send tcp 10.1.1.10 1234 "myCommand" equals "myReply") {
    //success
} else {
    //error
}

if (send tcp 10.1.1.10 1234 "6D792068657820636F6D6D616E64" hex equals "6D792068657820666565646261636B") {
    //success
} else {
    //error
}

set var myVal 10
send tcp 10.1.1.10 1234 ("set vol "" & (get var myVal) & ""\x0D")
set vol "10"\x0D

send tcp 10.1.1.10 1234 "set vol ""20""\x0D"
set vol "20"\x0D

send tcp 10.1.1.10 1234 "sourceStatus" ...
    contains "source01" action (set ui_button <<ui_name>> "myBtn01" position down) ...
    contains "source02" action (set ui_button <<ui_name>> "myBtn02" position down) ...
    contains "source03" action (set ui_button <<ui_name>> "myBtn03" position down) ...
    actionelse (set ui_button <<ui_name>> "myBtnGroup" position up)

send tcp 10.1.1.10 1234 "state" exact "s01" actionelse (set ui_button "myUI" "myBtn01" position up)

send tcp 10.1.1.10 1234 "command" reply
set ui_label <<ui_name>> "myLbl" text <<send_response>>

send tcp 10.1.1.10 1234 "getVolume" reply
set ui_slider <<ui_name>> "mySlidr" value (val(<<send_response>>))
```

5.2.8 Response Examples

```
send tcp success<cr>
send tcp success [<feedback>]<cr>
send tcp success [disconnected]<cr>

send tcp error [incomplete]<cr>
send tcp error [invalid parameter]<cr>
send tcp error [invalid format]<cr>
send tcp error [device '<address>' not found]<cr>
send tcp error [<feedback>]<cr>
send error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

5.3 Command send telnet (future

functionality pending)

5.3.1 Method

```
send telnet [key:<security_key>] <address> <port> [username "<user_search>" "<user_send>"] ...  
[password "<pass_search>" "<pass_send>"] "<command>" ...  
["<feedback_string>" [<action> (<process>)]]<cr>
```

5.3.2 Description

The command **send telnet** provides a seamless integration with any TCP Telnet controllable device.

5.3.3 Arguments

argument	type	contains	
<address>	string	TCP IP address	
<port>	integer	TCP port	
username	string	Optional keyword indicating username login details to follow	
<user_search>	string	Username prompt	
<user_send>	string	Username to send	
password	string	Optional keyword indicating password login details to follow	
<pass_search>	string	Password prompt	
<pass_send>	string	Password to send	
<command>	string	Command string to send to TCP Telnet device	
<feedback>	string	Optional feedback Keyword reply equals contains	
reply	string	Return with response string	
equals	string	<feedback_string>	Complete response string
contains	string	<feedback_string>	Partial response string
<action>	string	Optional equals or contains action keyword action actionelse	
(<process>)	string	Command line wrapped with brackets	

5.3.4 Notes

- The TCP Telnet device must be accessible by the controller.
- To send hexadecimal bytes add \x before each byte. \x0D for carriage return.
- The <feedback> option uses keywords **reply**, **equals** or **contains** to set the type of feedback required. To receive a response only, use **reply**. For comparison with the specified <feedback_string> use **equals** for an exact match, or **contains** for a match within the response.
- <feedback_string> contains the expected response string.
- <user_search> or <pass_search> cannot be blank strings.
- Use double quotes "" within a "wrapped" string to send a quotation mark ".
`send telnet <address> <port> "wrap this value "10"\x0D"<cr>` will send *wrap this value "10"<cr>*
Blue are command wrappers for "<string>", Green are trigger "wrappers" with Black wrappers within "<string>".
- Use keyword **[disconnect]** in place of <command> to terminate the connection.
- The TCP connection will be maintained until disconnected.
- Use multiple feedback **contains** | **exact** options with **action** for status updates to perform UI functions as a result of the response received.
- **action** is a success response while **actionelse** is an error response.
- System variable <<send_response>> will be set with the response string.

COMMAND GUIDE V1.3.8.0

5.3.5 Response

<feedback> null	success ● establishing a TCP connection ● able to login error ● not able to establish a TCP connection ● not able to login
<feedback> reply	success ● establishing a TCP connection ● able to login ○ received string included in the send telnet command success response error ● not able to establish a TCP connection ● not able to login
<feedback> contains	success ● establishing a TCP connection ● able to login ● <feedback_string> found in the received string error ● not able to establish a TCP connection ● not able to login ● <feedback_string> not found in the received string ○ received string included in the send telnet command error response
<feedback> exact	success ● establishing a TCP connection ● able to login ● <feedback_string> matching the received string error ● not able to establish a TCP connection ● not able to login ● <feedback_string> not matching the received string ○ received string included in the send telnet command error response

5.3.6 Examples

```

send telnet 10.1.1.10 23 "command string"<cr>
send telnet 10.1.1.10 23 "command string\x0D\x0A" reply<cr>
send telnet 10.1.1.10 23 "command string" contains "response string"<cr>
send telnet 10.1.1.10 23 "command string" equals "response string\x0D\x0A"<cr>
send telnet 10.1.1.10 23 username "User:" "Name" "command string" reply<cr>
send telnet 10.1.1.10 23 username "User:" "Name" password "Password:" "Password" "command string"<cr>
send telnet 10.1.1.10 23 password "Password:" "Password" "set vol ""10""\x0D"<cr>
send telnet 10.1.1.10 23 [disconnect]<cr>
send telnet key:abc123 10.1.1.10 23 "command string"<cr>

send telnet 10.1.1.10 23 "sourceStatus" ...
  contains "source01" action (set ui_button "myUI" "myBtn01" position down) ...
  contains "source02" action (set ui_button "myUI" "myBtn02" position down) ...
  contains "source03" action (set ui_button "myUI" "myBtn03" position down) ...
  actionelse (set ui_button "myUI" "myBtnGroup" position up)<cr>

send telnet 10.1.1.10 23 "sourceStatus" contains "source01" ...
  actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send telnet 10.1.1.10 23 "sourceStatus" exact "source01" ...
  action (set ui_button "myUI" "myBtn01" position down) ...
  actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send telnet 10.1.1.10 23 "sourceStatus" exact "source01" ...
  actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

```

COMMAND GUIDE V1.3.8.0

5.3.7 Preset Examples

```
send telnet 10.1.1.10 23 username "login:" "MyLogin" password "Password:" "MyPassword" "myCommand"

if (send telnet 10.1.1.10 23 username "login:" "MyLogin" password "Password:" "MyPassword" "myCommand" contains "myReply") {
    //success
} else {
    //error
}

send telnet 10.1.1.10 23 "sourceStatus" ...
    contains "source01" action (set ui_button <<ui_name>> "myBtn01" position down) ...
    contains "source02" action (set ui_button <<ui_name>> "myBtn02" position down) ...
    contains "source03" action (set ui_button <<ui_name>> "myBtn03" position down) ...
    actionelse (set ui_button <<ui_name>> "myBtnGroup" position up)

send telnet 10.1.1.10 23 "state" exact "s01" actionelse (set ui_button "myUI" "myBtn01" position up)

send telnet 10.1.1.10 23 "command" reply
set ui_label <<ui_name>> "myLbl" text <<send_response>>

send telnet 10.1.1.10 23 "getVolume" reply
set ui_slider <<ui_name>> "mySlidr" value (val(<<send_response>>))
```

5.3.8 Response Examples

```
send telnet success<cr>
send telnet success [<feedback>]<cr>
send telnet success [disconnected]<cr>

send telnet error [incomplete]<cr>
send telnet error [invalid parameter]<cr>
send telnet error [invalid format]<cr>
send telnet error [invalid login]<cr>
send telnet error [device '<address>' not found]<cr>
send telnet error [<feedback>]<cr>
send error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

5.4 Command send udp (future functionality pending)

5.4.1 Method

```
send udp [key:<security_key>] <address> <port_send> [<port_receive>] "<command>" [<protocol>] ...  
[<feedback> ["<feedback_string>" [<action> (<process>)]]]<cr>
```

5.4.2 Description

The command **send udp** provides a seamless integration with any UDP controllable device.

5.4.3 Arguments

argument	type	contains	
<address>	string	UDP IP address	
<port_send>	integer	Port to send data	
<port_receive>	integer	Optional port to receive data when using <feedback>	
<command>	string	Command string to send to UDP device	
<protocol>	string	Optional keyword ascii (default) hex	
<feedback>	string	Optional feedback keyword reply equals contains	
reply	string	Return with response string	
equals	string	<feedback_string>	Complete response string
contains	string	<feedback_string>	Partial response string
<action>	string	Optional equals or contains action keyword action actionelse	
(<process>)	string	Command line wrapped with brackets	

5.4.4 Notes

- The UDP device must be accessible by the controller.
- To send hexadecimal bytes with protocol ASCII add \x before each byte. \x0D for carriage return.
- Optional keyword **hex** allows hexadecimal bytes to be sent and received without the need for \x.
- The <feedback> option uses keywords **reply**, **equals** or **contains** to set the type of feedback required. To receive a string only, use **reply**. For comparison with the specified <feedback_string> use **equals** for an exact match, or **contains** for a match within the string.
- <feedback_string> contains the expected response string.
- Use double quotes "" within a "wrapped" string to send a quotation mark ".
- `send udp <address> <port_send> "wrap this value ""10""\x0D"<cr>` will send `wrap this value "10"<cr>`
- Blue are command wrappers for "<string>", Green are trigger "wrappers" with Black wrappers within "<string>".
- Use keyword [**disconnect**] in place of <command> to terminate the connection.
- The UDP connection will be maintained for 10 seconds before automatically dropping.
- IP range 224.0.0.0 - 239.255.255.255 will always be treated as Multicast and join the Multicast group.
- Use multiple feedback **contains** | **exact** options with **action** for status updates to perform UI functions as a result of the response received.
- **action** is a success response while **actionelse** is an error response.
- System variable <<send_response>> will be set with the response string.

COMMAND GUIDE V1.3.8.0

5.4.5 Response

<feedback> null	success ● establishing a UDP connection error ● not able to establish a UDP connection
<feedback> reply	success ● establishing a UDP connection ○ received string included in the send udp command success response error ● not able to establish a UDP connection
<feedback> contains	success ● establishing a UDP connection ● <feedback_string> found in the received string error ● not able to establish a UDP connection ● <feedback_string> not found in the received string ○ received string included in the send udp command error response
<feedback> exact	success ● establishing a UDP connection ● <feedback_string> matches the received string error ● not able to establish a UDP connection ● <feedback_string> not matching the received string ○ received string included in the send udp command error response

5.4.6 Examples

```

send udp 255.255.255.255 10000 "an ascii string"<cr>
send udp 255.255.255.255 10000 "a mixed string\x0D"<cr>
send udp 255.255.255.255 10000 "\x61\x20\x48\x45\x58\x20\x73\x74\x72\x69\x6E\x67"<cr>
send udp 255.255.255.255 10000 10001 "command string" reply<cr>
send udp 10.1.1.10 10000 10001 "command string" contains "response string"<cr>
send udp 239.255.255.255 10000 10001 "command string" equals "response string\x0D"<cr>
send udp 255.255.255.255 10000 "6D792068657820636F6D6D616E64" hex reply<cr>
send udp 255.255.255.255 10000 10001 "6D792068657820636F6D6D616E64" hex equals "6D792068657820666565646261636B"<cr>
send udp 255.255.255.255 10000 "set vol ""10""\x0D"<cr>
send udp 239.255.255.255 10000 [disconnect]<cr>
send udp key:abc123 255.255.255.255 10000 "an ascii string"<cr>

send udp 255.255.255.255 10000 10001 "sourceStatus" ...
contains "source01" action (set ui_button "myUI" "myBtn01" position down) ...
contains "source02" action (set ui_button "myUI" "myBtn02" position down) ...
contains "source03" action (set ui_button "myUI" "myBtn03" position down) ...
actionelse (set ui_button "myUI" "myBtnGroup" position up)<cr>

send udp 255.255.255.255 10000 10001 "sourceStatus" contains "source01" ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send udp 255.255.255.255 10000 10001 "sourceStatus" exact "source01" ...
action (set ui_button "myUI" "myBtn01" position down) ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send udp 255.255.255.255 10000 10001 "sourceStatus" exact "source01" ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

```

COMMAND GUIDE V1.3.8.0

5.4.7 Preset Examples

```
send udp 255.255.255.255 10000 "myCommand"
send udp 255.255.255.255 10000 10001 "6D792068657820636F6D6D616E64" hex reply

if (send udp 255.255.255.255 10000 10001 "myCommand" equals "myReply") {
    //success
} else {
    //error
}

if (send udp 255.255.255.255 10000 10001 "6D792068657820636F6D6D616E64" hex equals "6D792068657820666565646261636B") {
    //success
} else {
    //error
}

send udp 255.255.255.255 10000 10001 "sourceStatus" ...
contains "source01" action (set ui_button <<ui_name>> "myBtn01" position down) ...
contains "source02" action (set ui_button <<ui_name>> "myBtn02" position down) ...
contains "source03" action (set ui_button <<ui_name>> "myBtn03" position down) ...
actionelse (set ui_button <<ui_name>> "myBtnGroup" position up)

send udp 255.255.255.255 10000 10001 "state" exact "s01" ...
actionelse (set ui_button "myUI" "myBtn01" position up)

send udp 255.255.255.255 10000 10001 "command" reply
set ui_label <<ui_name>> "myLbl" text <<send_response>>

send udp 255.255.255.255 10000 10001 "getVolume" reply
set ui_slider <<ui_name>> "mySlidr" value (val(<<send_response>>))
```

5.4.8 Response Examples

```
send udp success<cr>
send udp success [<feedback>]<cr>
send udp success [disconnected]<cr>

send udp error [incomplete]<cr>
send udp error [invalid parameter]<cr>
send udp error [invalid format]<cr>
send udp error [invalid port]<cr>
send udp error [invalid multicast address]<cr>
send udp error [<feedback>]<cr>
send error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

5.5 Command send http (future functionality pending)

5.5.1 Method

send http [key:<security_key>] <method> <protocol> <host> [<port>] [<resource>] [headers "<header01>, <header02>...">] [username "<username>"] [password "<password>"] [body <body>]... [<feedback> ["<feedback_string>" [<action> (<process>)]]]<cr>

5.5.2 Description

The command **send http** provides a seamless integration with any HTTP RESTful controllable device.

5.5.3 Arguments

argument	type	contains	
<method>	string	get post put delete	
<protocol>	string	http:// or https://	
<host>	string	Domain or IP Address of the device	
<port>	integer	Device port if other than 80 for http or 443 for https	
<resource>	string	Absolute API path	
headers	string	Optional keyword indicating the following headers are to be applied	
username	string	Optional keyword indicating username login details to follow	
<username>	string	Username	
password	string	Optional keyword indicating password login details to follow	
<password>	string	Password Optionally for MD5 encryption wrap the password with md5(<password>)	
body		Optional keyword indicating body details to follow	
<body>	string	Body data string	
<feedback>	string	Optional feedback keyword equals contains	
equals	string	<feedback_string>	
contains	string	<feedback_string>	Complete response string
<action>	string	Optional equals or contains action keyword action actionelse	Partial response string
(<process>)	string	Command line wrapped with brackets	

5.5.4 Notes

- The HTTP device must be accessible by the controller.
- The <feedback> option uses keywords **equals** or **contains** to set the type of feedback required.
- For comparison with the specified <feedback_string> use **equals** for an exact match, or **contains** for a match within the response.
- <feedback_string> contains the expected response string.
- Use multiple feedback contains | exact options with action for status updates to perform UI functions as a result of the response received.
- action is a success response while actionelse is an error response.
- System variable <<send_response>> will be set with the response string.

COMMAND GUIDE V1.3.8.0

5.5.5 Response

<feedback> null	success [<i>response</i>] • status code 200 returned error • status code 200 not returned
<feedback> contains	success [<i>response</i>] • status code 200 returned • <feedback_string> found in the response string error [<i>message</i>] • status code 200 not returned error [<i>response</i>] • <feedback_string> not found in the response string
<feedback> exact	success [<i>response</i>] • status code 200 returned • <feedback_string> matches the response string error [<i>message</i>] • status code 200 not returned error [<i>response</i>] • <feedback_string> not matching the response string

5.5.6 Examples

```

<method> = put
<protocol> = http://
<host> = 169.254.1.70
<resource> = /api/host/modules/1/relays/logicals/1
headers = headers "application/json"
body = body {"type": "SPST", "state": "off"}

```

```

send http put http://169.254.1.70/api/host/modules/1/relays/logicals/1 headers "application/json"...
body {"type": "SPST", "state": "off"}<cr>

```

```

<method> = post
<protocol> = http://
<host> = 172.16.0.210
<port> = 3333
<resource> = /api/settings
username = username "User"
password = password "md5(Password)"
body = body {"index":1,"label":"Test Cam"}

```

```

send http post http://172.16.0.210:3333/api/settings username "User" password "md5(Password)" body ...
{"index":1,"label":"Test Cam"}<cr>

```

COMMAND GUIDE V1.3.8.0

5.5.6 Examples continued...

```
send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] {"state":"on"}<cr>

send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] body {"state":"on"}...
contains ""state":"on""<cr>

send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] body {"state":"on"}...
equals...
{"href":"/api/host/modules/1/relays/logicals/1",...
"type":"SPST","access":"RW","portMap":"1","state":"on"}<cr>

send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] username "user" password "pass"...
body {"state":"on"}<cr>

send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] username "user" password "md5(pass)"...
body {"state":"on"}<cr>

send http get http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers[Connection:close, Content-Type:application/json, Cache-Control:max-age=600]<cr>

send http get http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers[Connection:close, Content-Type:application/json, Cache-Control:max-age=600]...
contains ""state":"on""<cr>

send http get http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers[Connection:close, Content-Type:application/json, Cache-Control:max-age=600]...
equals...
{"href":"/api/host/modules/1/relays/logicals/1",...
"type":"SPST","access":"RW","portMap":"1","state":"on"}<cr>

send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] body {"state":"on"} ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send http get http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers[Connection:close, Content-Type:application/json, Cache-Control:max-age=600]...
contains ""state":"on"" ...
action (set ui_button "myUI" "myBtn01" position down) ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>

send http get http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers[Connection:close, Content-Type:application/json, Cache-Control:max-age=600]...
contains ""state":"on"" ...
actionelse (set ui_button "myUI" "myBtn01" position up)<cr>
```

COMMAND GUIDE V1.3.8.0

5.5.7 Preset Examples

```
send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] body {"state":"on"}

if not (send http put http://10.1.1.64/api/host/modules/1/relays/logicals/1...
headers [Content-Type:application/json, Content-Length:11] body{"state":"on"}...
contains "\"state\":\"on\"") {
    send notification "Device Error" "Some Device"
}
```

5.5.8 Response Examples

```
send http success [<response>]<cr>

send http error [invalid parameter]<cr>
send http error [invalid method '<method>']<cr>
send http error [invalid headers]<cr>
send http error [invalid parameter]<cr>
send http error [invalid url '<url>']<cr>
send http error [could not connect]<cr>
send http error [request timed out]<cr>
send error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

5.6 Command send notification

(pending future command)

5.6.1 Method

send notification [key:<security_key>] "<subject_text>" "<message_text>"<cr>

5.6.2 Description

The command **send message** provides the ability to send an email message on demand. The message uses the Notifications email Address.

5.6.3 Arguments

argument	type	contains
<subject_text>	string	The subject matter of the notification email
<message_text>	string	The notification email message

5.6.4 Notes

- Notification emails must be configured prior to use, refer User Manual 1.6.1 Email Settings.

5.6.5 Response

command	<space>	mode	<space>	status	<space>	value	terminator
send	<space>	notification	<space>	success / error [message]	<space>	<string>	<cr>

5.6.6 Examples

```
send notification "Warning" "something has happened"<cr>
send notification key:abc123 "User" "assistance required"<cr>
```

5.6.7 Preset Examples

```
send notification "Feedback" "Worked Perfectly"

if not (send tcp 10.1.1.10 1234 "myCommand" equals "myReply") {
    send notification "Device Error" "Some Device"
}
```

5.6.8 Response Examples

```
send notification success<cr>

send notification error [subject empty]<cr>
send notification error [message empty]<cr>
send notification error [failed to send]<cr>
send error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

6 Command preset

The preset commands are used to store and apply a series of commands available from this manual. Refer Appendix A - Preset logic for logic that can be applied within a preset.

Preset commands **add** | **delete** | **load** are not valid in a preset and can only be used to create, delete and execute presets via the API.

6.1 Command preset add

6.1.1 Method

```
preset add [key:<security_key>] <preset_name> <preset_data><cr>
```

6.1.2 Description

The command **preset add** is used to create and append commands to the specified preset.

6.1.3 Arguments

argument	type	contains
<preset_name>	string	Preset name
<preset_data>	string	Valid control command data

6.1.4 Notes

- The preset is executed with the **preset load** command.
- Command is not available from within a preset.

6.1.5 Response

command	<space>	mode	<space>	name	<space>	status	terminator
preset	<space>	add	<space>	<preset_name>	<space>	success / error [message]	<cr>

6.1.6 Examples

```
preset add preset1 send gc 169.254.1.70 4998 setstate,1:1,1<cr>
preset add key:abc123 preset1 send gc 169.254.1.70 4999 "My String"<cr>
```

6.1.7 Response Examples

```
preset add preset1 success<cr>

preset add error [incomplete]<cr>
preset error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

6.2 Command preset delay

6.2.1 Method

preset delay [key:<security_key>] <milliseconds>

6.2.2 Description

The command **preset delay** is used within a preset to add a delay between commands.

6.2.3 Arguments

argument	type	contains
<milliseconds>	integer	Delay time in milliseconds from 1 to 9999

6.2.4 Notes

- Command can only be used within a preset.

6.2.5 Response

None

6.2.6 Examples

```
preset delay 1000
preset delay key:abc123 500
```

6.2.7 Response Example

No return is given for a valid command

```
preset delay error [incomplete]<cr>
preset delay error [invalid milliseconds]<cr>
preset error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

6.3 Command preset delete

6.3.1 Method

```
preset delete [key:<security_key>] <preset_name><cr>
```

6.3.2 Description

The command **preset delete** is used to delete the specified preset.

6.3.3 Arguments

argument	type	contains
<preset_name>	string	Preset name

6.3.4 Notes

- Command is not available from within a preset.

6.3.5 Response

command	<space>	mode	<space>	name	<space>	status	terminator
preset	<space>	delete	<space>	<preset_name>	<space>	success / error [message]	<cr>

6.3.6 Examples

```
preset delete preset1<cr>  
preset delete key:abc123 preset1<cr>
```

6.3.7 Response Examples

```
preset delete preset1 success<cr>  
  
preset delete error [incomplete]<cr>  
preset delete error [preset '<preset_name>' not found]<cr>  
preset error [invalid mode]<cr>
```

COMMAND GUIDE V1.3.8.0

6.4 Command preset load

6.4.1 Method

```
preset load [key:<security_key>] <preset_name> [<delay>]<cr>
```

6.4.2 Description

The command **preset load** is used to apply stored commands within the specified preset.

6.4.3 Arguments

argument	type	contains
<preset_name>	string	Preset name
<delay>	integer	Optional delay in minutes from 1 to 60 or -1 (optional)

6.4.4 Notes

- The preset is created with the **preset add** command or directly via the UI.
- Optional **<delay>** is used to delay a preset for the specified amount of time in minutes. The delay is reset each time the command is executed and **-1** will terminate the command.
- Command is not available from within a preset.

6.4.5 Response

command	<space>	mode	<space>	name	<space>	status	terminator
preset	<space>	load	<space>	<preset_name>	<space>	success / error [message]	<cr>

6.4.6 Examples

```
preset load preset1<cr>
preset load preset1 30<cr>
preset load preset1 -1<cr>
preset load key:abc123 preset1<cr>
```

6.4.7 Response Examples

```
preset load preset1 success<cr>
preset load preset1 delayed<cr>

preset load error [incomplete]<cr>
preset load error [invalid delay]<cr>
preset load error [preset '<preset_name>' not found]<cr>
preset load preset1 error [...]<cr>
preset error [invalid mode]<cr>
```

7 Preset Logic Commands

Within presets the following logic commands can be used:

- **instr()** Boolean use returns as false when **<string>** does not contain **<searchString>** and returns true when **<string>** does contain **<searchString>**.
Integer use returns as 0 when **<string>** does not contain **<searchString>** and returns the start position of **<searchString>** within **<string>** from 1.
Optional use of **<offset>** can be used to add or subtract from the resulting index.
- **instrrev()** Boolean use returns as false when **<string>** does not contain **<searchString>** and returns true when **<string>** does contain **<searchString>**.
Integer use returns as 0 when **<string>** does not contain **<searchString>** and returns the end position of **<searchString>** within **<string>** from 1.
Optional use of **<offset>** can be used to add or subtract from the resulting index.
- **replace()** Replaces a string with another string.
- **strlen()** Returns the length of the specified **<string>**.
- **substr()** Will extract a string from a string using a starting index and length.
- **substring()** Will extract a string from a string using a starting and ending index.
- **trim()** Removes any hex bytes after the last ASCII character, like terminators **<cr>** from the end of the string.
- **val()** Extract a number from a string.
- **dec()** Decrement the integer value of **<variable>** by **<optional_amount>**, otherwise 1 used as default.
- **inc()** Increment the integer value of **<variable>** by **<optional_amount>**, otherwise 1 used as default.
- **loop while()** Used to repeat a script block.
- **select case()** Used in place of multiple if / elseif statements.
- **return()** Used to override the return status of a preset and optionally terminate the presets execution.
- **suspend()** Used to delay the operation of a command while not delaying the presets execution.

COMMAND GUIDE V1.3.8.0

7.1 Preset Command instr()

7.1.1 Method

```
( instr( <string>, <searchString>, <optional_result_offset> ) )
```

7.1.2 Description

Used to retrieve an Integer specifying the position of the first occurrence of a string **<searchString>** within another string **<string>**. Can also be used check if **<searchString>** is present in **<string>**.

7.1.3 Arguments

argument	type	contains
<string>	string	String as "wrapped" text or string reference
<searchString>	string	String as "wrapped" text or string reference to find in <string>
<optional_result_offset>	integer	Optional value added or subtracted from the result

7.1.4 Notes

- Useful to find if a string exists in another, or within substr() and substring() finding a string index.

7.1.5 Response

type	contains
integer	The position of the first occurrence of <searchString> within <string>
boolean	Returns true when <searchString> is present in <string> else return false

7.1.6 Examples

```
set var myVar "LED_LIGHTING,1:3,25,50"

if (instr((get var myVar),"LIGHTING")) {
    //myVar contains "LIGHTING"
} else {
    // myVar does not contain "LIGHTING"
}

set var myVarIndex (instr((get var myVar),"LIGHTING"))
myVarIndex = 5

set var myVarIndex (instr((get var myVar),"ERROR"))
myVarIndex = 0

set var myVarIndex (instr((get var myVar),"1:3",4))
myVarIndex = 18
```

COMMAND GUIDE V1.3.8.0

7.2 Preset Command instrrev()

7.2.1 Method

```
( instrrev( <string>, <searchString>, <optional_result_offset> ) )
```

7.2.2 Description

Used to retrieve an Integer specifying the position of the last occurrence of a string **<searchString>** within another string **<string>**. Can also be used check if **<searchString>** is present in **<string>**.

7.2.3 Arguments

argument	type	contains
<string>	string	String as "wrapped" text or string reference
<searchString>	string	String as "wrapped" text or string reference to find in <string>
<optional_result_offset>	integer	Optional value added or subtracted from the result

7.2.4 Notes

- Useful to find an index of a string prior to another string.

7.2.5 Response

type	contains
integer	The position of the last occurrence of <searchString> within <string>
boolean	When <searchString> is present in <string> then true else false

7.2.6 Examples

```
set var myVar "LED_LIGHTING,1:3,25,50"

if (instrrev((get var myVar),"LIGHTING")) {
    //myVar contains "LIGHTING"
} else {
    // myVar does not contain "LIGHTING"
}

set var myVarIndex (instrrev((get var myVar),"LIGHTING"))
myVarIndex = 5

set var myVarIndex (instrrev((get var myVar),"ERROR"))
myVarIndex = 0

set var myVarIndex (instrrev((get var myVar),"1:3",4))
myVarIndex = 18
```

7.2.6 Examples continued...

In this example addrbook 1 entry will be removed by name:

```
set var addrbook "...
  addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0A
  addrbook 1. "Polycom Demo 2" sip_spd:384 sip_num:polycomg7500@polycom.com\x0D\x0A
  addrbook 2. "Polycom Demo 3" phone_num:1.512.5121212\x0D\x0A
  addrbook all done\x0D\x0A"

set var addrbookName "Polycom Demo 2"

set var indexStart (instr((get var addrbook), (get var addrbookName)))
indexStart = 101

set var indexEnd (instr((substring((get var addrbook), (get var indexStart))), "\x0D\x0A", 7))
indexEnd = 69

(inc(indexEnd, (get var indexStart)))
indexEnd = 170

set var addrbookName (substring((get var addrbook), 0, (get var indexEnd)))
addrbookName = addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0Aaddrbook 1. "Polycom
Demo 2" sip_spd:384 sip_num:polycomg7500@polycom.com\x0D\x0A

set var addrbookName (substring((get var addrbookName), instrrev((get var addrbookName), ...
"addrbook")))
instrrev inx = 88
addrbookName = addrbook 1. "Polycom Demo 2" sip_spd:384 sip_num:polycomg7500@polycom.com\x0D\x0A

set var addrbook (replace((get var addrbook), (get var addrbookName), ""))
addrbookName = addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0Aaddrbook 2. "Polycom
Demo 3" phone_num:1.512.5121212\x0D\x0Aaddrbook all done\x0D\x0A
```

COMMAND GUIDE V1.3.8.0

7.3 Preset Command replace()

7.3.1 Method

```
( replace( <string>, <searchString>, <replaceString> ) )
```

7.3.2 Description

Used to replace a substring with another substring or remove a substring in a string.

7.3.3 Arguments

argument	type	contains
<string>	string	String reference or string as "wrapped" text
<searchString>	string	String as "wrapped" text or string reference to find in <string>
<replaceString>	string	String as "wrapped" text or string reference to replace <searchString> in <string>

7.3.4 Notes

- Useful to create an expected feedback response string derived from the command being sent.

7.3.5 Response

type	contains
string	<string> in which <searchString> has been replaced with <replaceString>

7.3.6 Examples

```
set var myVar "testing:123"  
set var myVar (replace((get var myVar),":",""))  
myVar = testing123
```

```
(replace("abcdef","cd","")) = "abef"
```

```
(replace("this command value is 123",123,"")) = this command value is ""
```

example command = set 3>4

example response = setting ok 3>4

```
set var myCmd "set 3>4"  
set var myRes (replace((get var myCmd),"set","setting ok"))  
send tcp 10.1.1.10 1234 ((get var myCmd) equals (get var myRes))
```

example command = set 3>4

example response = setting 3>4 ok

```
set var myCmd "set 3>4"  
set var myRes ((replace((get var myCmd),"set","setting")) & "ok")  
send tcp 10.1.1.10 1234 ((get var myCmd) equals (get var myRes))
```

COMMAND GUIDE V1.3.8.0

7.4 Preset Command strlen()

7.4.1 Method

(strlen(<string>))

7.4.2 Description

Used to retrieve the length of a string <string>.

7.4.3 Arguments

argument	type	contains
<string>	string	String as "wrapped" text or string reference

7.4.4 Notes

- Useful to find if a string is empty or not.

7.4.5 Response

type	contains
integer	Length of <string>
boolean	If strlen(<string>) > 0 then true else false

7.4.6 Examples

```
strlen("0123456789")= 10

if ((strlen((get var myVar))) > 9) {
    //myVar contains 10 or more characters
} else {
    // myVar does not contain 10 or more characters
}

if (strlen((get var myVar)) {
    //myVar contains 1 or more characters
} else {
    // myVar contains 0 characters
}

set var myVar "0123456789"
set var myVarLength (strlen((get var myVar)))
myVarLength = 10
```

COMMAND GUIDE V1.3.8.0

7.5 Preset Command substr()

7.5.1 Method

```
( substr( "<string>", <startIndex>, <optional_numOfCharacters> ) )
```

7.5.2 Description

Used to retrieve a substring from a string **<string>**. The substring starts at a specified character position **<startIndex>** and continues for an optional number of characters **<optional_numOfCharacters>**.

7.5.3 Arguments

argument	type	contains
<string>	string	String as "wrapped" text or string reference
<startIndex>	integer	One-based starting character position of a substring in <string>
<optional_numOfCharacters>	integer	Optional number of character from <startIndex>

7.5.4 Notes

- substr()** is useful when the length of the required string is known whereas **substring()** is useful when the ending position of the required string is known.

7.5.5 Response

type	contains
string	A substring of <string> that begins at <startIndex> and ends at optional number of characters <optional_numOfCharacters> or the end of <string>
string	An empty string if <startIndex> is greater than or equal to (>=) the length of the <string>

7.5.6 Examples

```
set var myVar "LED_LIGHTING,1:3,25,50"
set ui_label myUI myLbl text (substr((get var myVar),1))
lbl01 text = "LED_LIGHTING,1:3,25,50"

set ui_label myUI myLbl text (substr((get var myVar),18))
lbl01 text = " 25,50"

set ui_label myUI myLbl text (substr((get var myVar),18,2))
lbl01 text = "25"

(substr("abcdef", 4)) = "def"

(substr("abcdef", 4, 1)) = "d"
```

COMMAND GUIDE V1.3.8.0

7.6 Preset Command substring()

7.6.1 Method

(substring(<string>, <startIndex>, <optional_endIndex>))

7.6.2 Description

Used to retrieve a substring from a string **<string>**. The substring starts at a specified character position **<startIndex>** and continues to the optional end position **<optional_endIndex>**.

Using a negative **<optional_endIndex>** value such as -1 will subtract the value from the length of **<string>**.

7.6.3 Arguments

argument	type	contains
<string>	string	String as "wrapped" text or string reference
<startIndex>	integer	One-based starting character position of a substring in <string>
<optional_endIndex>	integer	Optional end of <string> from <startIndex>

7.6.4 Notes

- **substring()** is useful when the ending position of the required string is known otherwise **substr()** is useful when the length of the required string is known.

7.6.5 Response

type	contains
string	A substring of <string> that begins at <startIndex> and ends at optional position <optional_endIndex> , or the end of <string>
string	An empty string if <startIndex> is greater than or equal to ($\geq$) the length of the <string>

7.6.6 Examples

```
set var myVar "LED_LIGHTING,1:3,25,50"
set ui_label myUI myLbl text (substring((get var myVar),1))
lbl01 text = "LED_LIGHTING,1:3,25,50"

set ui_label myUI myLbl text (substring((get var myVar),18,20))
lbl01 text = "25"

(substring("abcdef",4)) = "def"
(substring("abcdef",4,5)) = "d"

(substring("abcdef",0,-1)) = "abcde"

set var myVar "0123456789"
(substring((get var string),0)) = "0123456789"
(substring((get var string),1)) = "0123456789"
(substring((get var string),0,10)) = "012345678"
(substring((get var string),1,10)) = "012345678"
(substring((get var string),0,-1)) = "012345678"
(substring((get var string),1,-1)) = "012345678"
(substring((get var string),0,-2)) = "01234567"
(substring((get var string),1,-2)) = "01234567"
(substring((get var string),0,(strlen(get var string)))) = "012345678"
(substring((get var string),0,(strlen(get var string)) -1)) = "01234567"
```

COMMAND GUIDE V1.3.8.0

7.7 Preset Command trim()

7.7.1 Method

```
( trim ( <string> ) )
```

7.7.2 Description

Used to remove trailing hex bytes such as carriage returns <cr> \x0D and line feed <lf> \x0A from <string>.

7.7.3 Arguments

argument	type	contains
<string>	string	String reference

7.7.4 Notes

- Provides a quick method to remove feedback response terminators when creating <feedback> strings for **equals** or **contains**.

7.7.5 Response

type	contains
string	A substring of <string> stripped of trailing hex bytes

7.7.6 Examples

```
if (trim(send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" reply) == "state,1:1,1") {  
    set ui_label myUI myLbl text "good"  
} else {  
    set ui_label myUI myLbl text "bad"  
}
```

```
set var myVar (send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" reply)  
if (trim(get var myVar) == "state,1:1,1") {  
    set ui_label myUI myLbl text "good"  
} elseif (trim(get var myVar) == "state,1:1,0") {  
    set ui_label myUI myLbl text "bad"  
} else {  
    set ui_label myUI myLbl text "error"  
}
```

```
set var myVar (trim(send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" reply))  
if (get var myVar == "state,1:1,1") {  
    set ui_label myUI myLbl text "good"  
} elseif (get var myVar == "state,1:1,0") {  
    set ui_label myUI myLbl text "bad"  
} else {  
    set ui_label myUI myLbl text "error"  
}
```

```
(trim("abcdef")) = 8
```

COMMAND GUIDE V1.3.8.0

7.8 Preset Command val()

7.8.1 Method

```
( val( <string>, <optional_offset>, <optional_default> ) )
```

7.8.2 Description

Used to retrieve the numbers contained in a string from an optional starting position.
The function stops reading the string at the first character that is not recognised as a number.

7.8.3 Arguments

argument	type	contains
<string>	string	String reference
<optional_offset>	integer	Optional one-based starting character position in <string>
<optional_default>	integer	Option Response other than 0 if <string> does not contain numbers

7.8.4 Notes

- Useful in obtaining an integer value from a feedback **reply** string then setting a UI level slider and or level indicator with the value obtained. Allowing it easy to obtain an amplifier volume level via TCP and set a UI level slider with only one process.

7.8.5 Response

type	contains
integer	The first number contained in <string> from <optional_offset>
integer	0 or <optional_default> if <string> does not contain numbers

7.8.6 Examples

```
set var myVar (val("volume"))
myVar = 0

set var myVar (val("volume",1,-1))
myVar = -1

set var myVar (val("volume",1,1))
myVar = 1

set var myVar (val("volume:10"))
myVar = 10

set var myVar "volume:100"
set var myVar (val(get var myVar))
myVar = 100

set var myVar "LED_LIGHTING,1:3,50,100"
set var myVar (val(get var myVar))
myVar = 1
set var myVar (val(get var myVar),18)
myVar = 50
set var myVar (val(get var myVar),21)
myVar = 100

(val("this command value is "123")) = 123
```

Obtain an amplifier volume status and set UI level slider in 3 processes:

```
set var myVol (send tcp 10.1.1.10 1234 "get volume\x0D" reply)
myVol = get volume success 50\x0D
set var myVol (val(get var myVol))
myVol = 50
set ui_slider myUI mySlidr value (get var myVol)
```

Obtain an amplifier volume status and set UI level slider in 2 processes:

```
set var myVol (val((send tcp 10.1.1.10 1234 "get volume\x0D" reply)))
myVol = 50
set ui_slider myUI mySlidr value (get var myVol)
```

Obtain an amplifier volume status and set UI level slider in **1** process:

```
set ui_slider myUI mySlidr value (val((send tcp 10.1.1.10 1234 "get volume\x0D" reply)))
mySlidr = 50
```

COMMAND GUIDE V1.3.8.0

7.9 Preset Command dec()

7.9.1 Method

dec(<variable>, <optional_amount>)

7.9.2 Description

Used to decrement a variable <variable> integer value by 1 or optional amount <optional_amount>. Also provides the method to subtract two integer variable values.

7.9.3 Arguments

argument	type	contains
<variable>	string	Name of variable containing an integer value
<optional_amount>	integer	Optional amount to decrement <variable> other than 1

7.9.4 Notes

- dec()** also provides the only means to subtract two numbers.
 $A - B = C$
 $C = \text{dec}(A,B)$

7.9.5 Response

type	contains
integer	Value of <variable> minus 1
integer	Value of <variable> minus <optional_amount>

7.9.6 Examples

```
set var myVar 10
dec(myVar)
myVar = 9
dec(myVar, 2)
myVar = 8
```

```
indexStart = 11
indexEnd = 12
(dec(indexEnd, (get var indexStart)))
indexEnd = 1
```

```
myStr = "indexStart [11]"
indexEnd = 12
(dec(indexEnd, (val((get var myStr)))))
indexEnd = 1
```

COMMAND GUIDE V1.3.8.0

7.10 Preset Command inc()

7.10.1 Method

inc(<variable>, <optional_amount>)

7.10.2 Description

Used to increment a variable <variable> integer value by 1 or optional amount <optional_amount>. Also provides the method to add two integer variable values together.

7.10.3 Arguments

argument	type	contains
<variable>	string	Name of variable containing an integer value
<optional_amount>	integer	Optional amount to increment <variable> other than 1

7.10.4 Notes

- inc() also provides the only means to add two numbers together.
 $A + B = C$
 $C = \text{inc}(A,B)$

7.10.5 Response

type	contains
integer	Value of <variable> plus 1
integer	Value of <variable> plus <optional_amount>

7.10.6 Examples

```
set var myVar 10
inc(myVar)
myVar = 11
inc(myVar,2)
myVar = 12
```

```
indexStart = 11
indexEnd = 12
(inc(indexEnd, (get var indexStart)))
indexEnd = 23
```

```
myStr = "indexStart [11]"
indexEnd = 12
(inc(indexEnd, (val((get var myStr)))))
indexEnd = 23
```

COMMAND GUIDE V1.3.8.0

7.11 Preset Command loop while()

(pending future command)

7.11.1 Method

```
loop while ( <object> <condition> <value> ) {  
    <action>  
}
```

7.11.2 Description

Used to cycle through a section of code while **<object>** complies with the **<condition>** of **<value>**.

7.11.3 Arguments

argument	type	contains
<object>	string	Reference to a variable or UI element state position text
<condition>	string	== != < > <= >=
<value>	string integer	Reference to a variable or UI element state position text Or Boolean, Integer, String as value

7.11.4 Notes

- Useful for repeating the same process to multiple UI elements or processing a string data.

7.11.5 Response

NONE

7.11.6 Examples

```
set var x 0  
loop while (x < 10) {  
    inc(x)  
    set var myBtn ("btn_" & (get var x))  
    if ((strlen((get ui_button <<ui_name>> (get var myBtn) text up))) > 0) {  
        set ui_button <<ui_name>> (get var myBtn) state enabled  
    }  
}
```

```
set var x 1  
loop while (x != (get var totalLoops)) {  
    set var myBtn ("btn_" & (get var x))  
    set ui_button <<ui_name>> (get var myBtn) text both ""  
    inc(x)  
}
```

```
loop while (<<button_position>> == "down") {  
    <action>  
}
```

7.12 Preset Command select case()

7.12.1 Method

```
select case ( <expression> ) {  
  case "<value>"  
    <action>  
  case else  
    <action>  
}
```

```
select case ( <expression> ) {  
  case "<value01>,<value02>..."  
    <action>  
  case else  
    <action>  
}
```

7.12.2 Description

Executes one of several groups of statements, depending on the value of a *<expression>*.

Select case is very useful for applying different functions for different results and replacing multiple if / elseif conditional statements.

All case conditions must be "wrapped" except case else.

7.12.3 Arguments

argument	type	contains
<expression>	string	Expression or value by reference

7.12.4 Notes

- Within a **select case()** statement **select case()** can only be used again within **case else**.

7.12.5 Response

Selected case result.

7.12.6 Examples

```
select case (<expression>) {
  case "1"
    <action>
  case "2"
    if (<value>) {
      <action>
    } elseif (<value>) {
      <action>
    } else {
      <action>
    }
  case else
    <action>
    select case (<value>) {
      case "1"
        <action>
      case "2"
        if (<value>) {
          <action>
        } elseif (<value>) {
          <action>
        }
      case else
        <action>
    }
}
```

Replace the following if / elseif statement with the following select case statement.

```
set var myVar <value>
if ((get var myVar) == "1") {
  <action>
} elseif ((get var myVar) == "2") {
  <action>
} elseif ((get var myVar) == "3") {
  <action>
} elseif ((get var myVar) == "4") {
  <action>
} elseif ((get var myVar) == "5") {
  <action>
}

set var myVar <value>
select case (get var myVar) {
  case "1"
    <action>
  case "2"
    <action>
  case "3"
    <action>
  case "4"
    <action>
  case "5"
    <action>
}
```

7.12.6 Examples continued...

```
select case (get var myNames) {
  case ""David Smith""
    <action>
  case ""John Smith""
    <action>
  case ""Peter Smith""
    <action>
  case ""Michael Smith""
    <action>
}

select case (<<button_name>>) {
  case "btn01"
    <action>
  case "btn02"
    <action>
  case "btn03"
    <action>
}

select case (<<button_position>>) {
  case "up"
    <action>
  case else
    <action>
}

select case (<<slider_name>>) {
  case "sldr01"
    <action>
  case "sldr02"
    <action>
  case "sldr03"
    <action>
}

select case (<<slider_value>>) {
  case "0"
    <action>
  case "100"
    <action>
  case else
    <action>
}

select case (<<button_name>>) {
  case "btn01", "btn02", "btn03"
    <action>
  case "btn04"
    <action>
  case else
    <action>
}
```

COMMAND GUIDE V1.3.8.0

7.13 Preset Command return()

7.13.1 Method

return (<expression>, <optional_termination>)

7.13.2 Description

Used to optionally terminate a preset with a given result or override the preset result.

7.13.3 Arguments

argument	type	contains
<expression>	string	true false null
<optional_termination>	string	true false (default)

7.13.4 Notes

- The following are preset return and button glow behaviour options:

return(true)	Override the result as success with green button glow
return(false)	Override the result as error with red button glow
return(null)	Override the result as success with no button glow
return(true,false)	Override the result as success with green button glow
return(false,false)	Override the result as error with red button glow
return(null,false)	Override the result as success with no button glow
return(true,true)	Terminate preset at current location as success with green button glow
return(false,true)	Terminate preset at current location as error with red button glow
return(null,true)	Terminate preset at current location as success with no button glow

7.13.5 Response

As specified with <expression>.

7.13.6 Examples

```
return(true)
return(false)
return(null)

return(true,false)
return(false,false)
return(null,false)

return(true,true)
return(false,true)
return(null,true)
```

COMMAND GUIDE V1.3.8.0

7.14 Preset Command suspend()

7.14.1 Method

suspend (<process>, <sec>)

7.14.2 Description

Used to delay the operation of a command while not holding up and delaying the presets execution.

7.14.3 Arguments

argument	type	contains
<process>	string	Any valid preset command combination
<sec>	integer	Delay time in seconds 0, 0.5..30

7.14.4 Notes

- Useful in clearing messages that have been displayed, or flashing up help information.

7.14.5 Response

NONE

7.14.6 Examples

```
suspend((set ui_label <<ui_name>> lb101 text ((substr("****",0,(strlen((get var pin)))))), 0.5)
suspend(set ui_label <<ui_name>> lb101 text "", 30)
suspend(set ui_label <<ui_name>> lb101 text "", (get var delay))
suspend((set ui_label <<ui_name>> lb101 text (send tcp 10.1.1.10 1234 "getVol\x0D" reply), 10)
```

8 Preset Operators

Operators are symbols with defined functions to perform a particular action.

8.1 Preset Operator 'And' &

8.1.1 Method

`<string> & <string>`

8.1.2 Description

Used as a logical *And* to append strings together.

8.1.3 Examples

```
set var myVar (send tcp 10.1.1.10 1234 "\x00\x01" reply)
myVar = "OK"
send tcp 10.1.1.10 1234 ("TX: " & (get var myVar) & "\x0D")
string sent = "TX: OK\x0D"
```

8.2 Preset Operator 'AndAlso' &&

8.2.1 Method

`<boolean> && <boolean>`

8.2.2 Description

Used as a logical *AndAlso* for multiple boolean conditional statements.

8.2.3 Examples

```
if (get ui_button myUI Btn01 position == "down" && get ui_button myUI Btn02 position == "down") {
    send gc 169.254.1.70 4998 setstate,1:1,1
}
```

8.3 Preset Operator 'OrElse' ||

8.3.1 Method

<boolean> || <boolean>

8.3.2 Description

Used as a logical *OrElse* for multiple boolean conditional statements.

8.3.3 Examples

```
if (get ui_button myUI myBtn01 position == "down" || get ui_button myUI myBtn02 position == "down") {  
    send gc 169.254.1.70 4998 setstate,1:1,1  
}
```

8.4 Preset Operator 'EqualTo' ==

8.4.1 Method

<statement> == <condition>

8.4.2 Description

Used to compare numbers or strings to equal.

8.4.3 Examples

```
if (get ui_button myUI myBtn01 position == "down") {  
    <action>  
}  
  
if (get ui_slider myUI mySldr value == 50) {  
    <action>  
}
```

COMMAND GUIDE V1.3.8.0

8.5 Operator 'NotEqualTo' !=

8.5.1 Method

<statement> != <value>

8.5.2 Description

Used to compare numbers or strings to not equal.

8.5.3 Examples

```
if (get ui_button myUI myBtn position != "down") {  
    <action>  
}  
  
if (get ui_slider myUI mySldr value != 50) {  
    <action>  
}
```

8.6 LessThan <

8.6.1 Method

<statement> < <value>

8.6.2 Description

Used to compare one number is less than another.

8.6.3 Examples

```
if (get ui_slider myUI mySldr value < 50) {  
    <action>  
}
```

8.7 Operator 'GreaterThan' >

8.7.1 Method

<statement> > <value>

8.7.2 Description

Used to compare one number is greater than another.

8.7.3 Examples

```
if (get ui_slider myUI mySldr value > 50) {  
    <action>  
}
```

8.8 Operator 'LessThanOrEqualTo' <=

[ToC](#)

COMMAND GUIDE V1.3.8.0

8.8.1 Method

`<statement> <= <value>`

8.8.2 Description

Used to compare one number is less than or equal to another.

8.8.3 Examples

```
if (get ui_slider myUI mySldr value <= 50) {  
    <action>  
}
```

8.9 Operator 'GreaterThanOrEqualTo' >=

8.9.1 Method

`<statement> >= <value>`

8.9.2 Description

Used to compare one number is greater than or equal to another.

8.9.3 Examples

```
if (get ui_slider myUI mySldr value >= 50) {  
    <action>  
}
```

8.10 Operator 'Not' not

8.10.1 Method

`not <statement> <condition> <value>`

8.10.2 Description

Used to invert an boolean result.

8.10.3 Examples

```
if not (get ui_button myUI myBtn position == "down") {  
    <action>  
}
```

9 Preset System Variables (pending future variables)

Within a UI preset read-only system variables are available for active elements:

variable	availability
<<ui_name>>	Available from any UI associated preset while the service is enabled to return the UI name. Used in place of command parameter <ui_name> .
<<button_name>>	Available from a pressed button preset to return the active button name. Used in place of command parameter <button_name> .
<<button_position>>	Available from a pressed button preset to return the active button position. Used in place of command get ui_button <ui_name> <button_name> position .
<<slider_name>>	Available from a pressed level slider preset to return the active level slider name. Used in place of command parameter <slider_name> .
<<slider_value>>	Available from a pressed level slider preset to return the active level slider value. Used in place of command get ui_slider <ui_name> <slider_name> value
<<encoder_name>>	Available from a pressed split button preset to return the active Encoder name.
<<decoder_name>>	Available from a pressed split button preset to return the active Decoder name.
<<pin_code>>	Available from the submit button preset returning the entered pin code.
<<send_response>>	Available after a send command.

These allow presets to be used for multiple UI's without the need for renaming elements. Also simplifies coding by not using control commands to obtain the same details.

9.1 Preset System Variable

<<ui_name>>

9.1.1 Method

set ui_button <<ui_name>> <button_name> <function> <value>

9.1.2 Description

<<ui_name>> is used in place of any commands <ui_name>.

9.1.3 Notes

- <<ui_name>> is available in any UI element preset.

9.1.4 Response

type	contains
string	Name of UI as <ui_name>

9.1.5 Examples

```
if (get ui_button <<ui_name>> myBtn state == "disabled") {  
    get ui_button <<ui_name>> myBtn state enabled  
}
```

9.2 Preset System Variable

<<button_name>>

9.2.1 Method

```
get ui_button <ui_name> <<button_name>> <function>
set ui_button <ui_name> <<button_name>> <function> <value>

if (<<button_name>> == "<button_name>") {
    ...
}

select case (<<button_name>>) {
    case "<button_name>"
        ...
    case "<button_name>"
        ...
}

send tcp <address> <port> <<button_name>>
```

9.2.2 Description

<<button_name>> is available from the buttons preset identifying the active button by name.

<<button_name>> can be used in the buttons preset in place of any commands <button_name>.

9.2.3 Notes

- When the button does not have a name then <<button_name>> will resolve as string **button_name**.
- The button name is set from the buttons settings.

9.2.4 Response

type	contains
string	Name of button when available else button_name

9.2.5 Examples

```
if (get ui_button myUI <<button_name>> state == "disabled") {  
    set ui_button myUI <<button_name>> state enabled  
}  
  
if (<<button_name>> == "Btn01") {  
    send tcp 10.1.1.1 1234 "Button 1 was pressed"  
}  
  
select case (<<button_name>>) {  
    case "Btn01"  
        send tcp 10.1.1.1 1234 "Button 1 was pressed"  
    case "Btn02"  
        send tcp 10.1.1.1 1234 "Button 2 was pressed"  
}  
  
send tcp 10.1.1.1 1234 "Button " & <<button_name>> & " was pressed"
```

9.3 Preset System Variable

<<button_position>>

9.3.1 Method

```
if (<<button_position>> == "<button_position>") {  
    ...  
}  
  
select case (<<button_position>>) {  
    case "up"  
        ...  
    case else  
        ...  
}  
  
send tcp <address> <port> <<button_position>>
```

9.3.2 Description

<<button_position>> is available from the buttons preset identifying the active buttons position.

<<button_position>> can be used in the buttons preset in place of command:

get ui_button <ui_name> <button_name> position

9.3.3 Notes

- <<button_position>> will return as either **up** or **down**.
- When <<button_position>> is unavailable then it will resolve as string **button_position**.

9.3.4 Response

type	contains
string	Button position as up or down when available else button_position

9.3.5 Examples

```
if (<<button_position>> == "up") {  
    set ui_button myUI myBtn position down  
}
```

```
select case (<<button_position>>) {  
    case "up"  
        send tcp 10.1.1.1 1234 "Button 1 position up"  
    case else  
        send tcp 10.1.1.1 1234 "Button 1 position down"  
}
```

```
send tcp 10.1.1.1 1234 ("Position of " & <<button_name>> & " is " & <<button_position>>)
```

9.4 Preset System Variable

<<slider_name>>

9.4.1 Method

```
get ui_button <ui_name> <<slider_name>> <function>
set ui_button <ui_name> <<slider_name>> <function> <value>

if (<<slider_name>> == "<slider_name>") {
    ...
}

select case (<<slider_name>>) {
    case "<slider_name>"
        ...
    case "<slider_name>"
        ...
}

send tcp <address> <port> <<slider_name>>
```

9.4.2 Description

<<slider_name>> is available from the level sliders preset identifying the active level slider by name.

<<slider_name>> can be used in the level sliders preset in place of any commands <slider_name>.

9.4.3 Notes

- When the slider does not have a name then <<slider_name>> will resolve as string **slider_name**.
- The level sliders name is set in the level sliders settings.

9.4.4 Response

type	contains
string	Name of level slider as <slider_name> when available else slider_name

9.4.5 Examples

```
if (get ui_slider myUI <<slider_name>> state == "disabled") {  
    set ui_slider myUI <<slider_name>> state enabled  
}  
  
if (<<slider_name>> == "Sldr01") {  
    send tcp 10.1.1.1 1234 "Level Slider 1 was moved"  
}  
  
select case (<<button_name>>) {  
    case "Sldr01"  
        send tcp 10.1.1.1 1234 "Level Slider 1 was moved"  
    case "Sldr01"  
        send tcp 10.1.1.1 1234 "Level Slider 2 was moved"  
}  
  
send tcp 10.1.1.1 1234 "Level Slider " & <<slider_name>> & " was moved"
```

9.5 Preset System Variable

<<slider_value>>

9.5.1 Method

```
if (<<slider_value>> == "<value>") {  
    <action>  
}  
  
select case (<<slider_value>>) {  
    case "0"  
        <action>  
    case else  
        <action>  
}  
  
send tcp <address> <port> <<slider_value>>
```

9.5.2 Description

<<slider_value>> is available from the level sliders preset identifying the active sliders value.

<<slider_value>> can be used in the buttons preset in place of command:

get ui_slider <ui_name> <slider_name> value

9.5.3 Notes

- <<slider_value>> will range from the slider **min** to **max** settings values.
- When <<slider_value>> is unavailable then it will resolve as string **slider_value**.

9.5.4 Response

type	contains
string	Level slider <value> when available else slider_value

9.5.5 Examples

```
if (<<slider_value>> == "0") {  
    set ui_label myUI myLbl text "volume mute"  
} else {  
    set ui_label myUI myLbl text ("volume " & <<slider_value>>)  
}  
  
select case (<<slider_value>>) {  
    case "0"  
        send tcp 10.1.1.1 1234 "set volume mute"  
    case else  
        send tcp 10.1.1.1 1234 ("set volume " & <<slider_value>>)  
}  
  
send tcp 10.1.1.1 1234 ("Value of " & <<slider_name>> & " is " & <<slider_position>>)
```

9.6 Preset System Variable

<<pin_code>> (pending future variable)

9.6.1 Method

```
if (<<pin_code>> == "<value>") {  
    <action>  
}  
  
select case (<<pin_code>>) {  
    case "0000"  
        <action>  
    case else  
        <action>  
}
```

9.6.2 Description

<<pin_code>> is available from the submit button preset returning the entered pin code.
When <<pin_code>> is unavailable then it will resolve to string **pin_code**.

9.6.3 Notes

- A preset must be assigned to the login page submit button.

9.6.4 Response

type	contains
integer	Entered numbers when available from submit button else pin_code

9.6.5 Examples

```
select case <<pin_code>> {  
    case "0001"  
        set ui_page <<ui_name>> page1  
    case "0002"  
        set ui_page <<ui_name>> page2  
    case else  
        return(false)  
}
```

9.7 Preset System Variable

<<encoder_name>>

9.7.1 Method

```
if (<<encoder_name>> == "<device_name>") {  
    <action>  
}  
  
select case (<<encoder_name>>) {  
    case "<device_name>"  
        <action>  
    case else  
        <action>  
}  
  
send tcp <address> <port> <<encoder_name>>
```

9.7.2 Description

<<encoder_name>> is available from the split buttons preset identifying the active Encoder device by name.

<<encoder_name>> can be used in the split buttons preset in place of <encoder_device_name>.

9.7.3 Notes

- When the split button does not have a device or there is no active Encoder button then <<encoder_name>> will resolve as string **encoder_name**.
- The Encoder name is set in the split button settings.

9.7.4 Response

type	contains
string	Active Encoder name as <encoder_device_name> when available else encoder_name

9.7.5 Examples

```
if not (join x " <<encoder_name>> & " " & <<decoder_name>> & "\x0D") contains "join xxx success") {  
    set ui_button <<ui_name>> <<button_name>> position up  
    return(false)  
}  
  
send tcp 10.1.1.10 1234 ...  
("SET " & <<encoder_name>> & " " & <<decoder_name>> & "\x0D\x0A")...  
equals (<<encoder_name>> & ">" & <<decoder_name>> & "\x0D\x0A")\x0D
```

9.8 Preset System Variable

<<decoder_name>>

9.8.1 Method

```
if (<<decoder_name>> == "<device_name>") {  
    <action>  
}  
  
select case (<<decoder_name>>) {  
    case "<device_name>"  
        <action>  
    case else  
        <action>  
}  
  
send tcp <address> <port> <<decoder_name>>
```

9.8.2 Description

<<decoder_name>> is available from the split buttons preset identifying the active Decoder device by name.

<<decoder_name>> can be used in the split buttons preset in place of <decoder_device_name>.

9.8.3 Notes

- When the split button does not have a device or there is no active Decoder button then <<decoder_name>> will resolve as string **decoder_name**.
- The Decoder name is set in the split button settings.

9.8.4 Response

type	contains
string	Active Decoder name as <decoder_device_name> when available else decoder_name

9.8.5 Examples

```
if not (join x " <<encoder_name>> & " " & <<decoder_name>> & "\x0D") contains "join xxx success") {  
    set ui_button <<ui_name>> <<button_name>> position up  
    return(false)  
}
```

```
send tcp 10.1.1.10 1234 ("SET IN " & <<encoder_name>> & " OUT " & <<decoder_name>> & "\x0D\x0A")
```

COMMAND GUIDE V1.3.8.0

9.9 Preset System Variable

<<send_response>> (pending future variable)

9.9.1 Method

```
send gc 169.254.1.70 4999 "command" reply
send tcp 10.1.1.10 1234 "command" reply
send telnet 10.1.1.10 23 "command" reply
send udp 255.255.255.255 1234 4321 "command" reply
```

```
if (<<send_response>> == "string") {
    <action>
}
```

```
select case (<<send_response>>) {
    case "string"
        <action>
    case else
        <action>
}
```

9.9.2 Description

<<send_response>> will retain the response string from a send command.

9.9.3 Notes

- A response string must be received otherwise <<send_response>> will return as an empty string.

9.9.4 Response

type	contains
string	External device response string

9.9.5 Examples

```
send gc 10.1.1.10 4999 "command" reply
set ui_label <<ui_name>> "myLbl" text <<send_response>>

send gc 10.1.1.10 4999 "getVolume" reply
set ui_slider <<ui_name>> "mySlidr" value (val(<<send_response>>))
```

```
send tcp 10.1.1.10 1234 "command" reply
set ui_label <<ui_name>> "myLbl" text <<send_response>>

send tcp 10.1.1.10 1234 "getVolume" reply
set ui_slider <<ui_name>> "mySlidr" value (val(<<send_response>>))
```

Appendix A – Preset logic

Basic logic can be applied within a preset to allow building some *smarts* into the system. All commands can be used as an argument with conditional statements.

The following syntax applies:

```
if (<value>) {
    <action>
    ...
} elseif (<value>) {
    <action> ...
} else {
    <action> ...
}

select case (<value>) {
    case "1"
        <action>
        ...
    case "text"
        <action>
        ...
    case "true"
        <action>
        ...
    case else
        <action>
}

select case (<value>) {
    case "1"
        <action>
    case "2"
        if (<value>) {
            <action>
        } elseif (<value>) {
            <action>
        } else {
            <action>
        }
    case else
        select case (<value>) {
            case "a"
                <action>
            case "b"
                if (<value>) {
                    <action>
                } elseif (<value>) {
                    <action>
                } else {
                    <action>
                }
            case else
                <action>
        }
}
```

Anything that needs to be resolved in a logic string must be wrapped in brackets (). Brackets are resolved in a logic string from the middle out. So in the following example, (get var myVar) is firstly replaced with the variable value. Next the variable value will be split with substr(). Finally the result of substr() is then used for the set ui_label command.

```
set ui_label myUI myLbl text (substr((get var myVar),1,4))
```

1st resolved set of brackets for get var = set ui_label myUI myLbl text (substr("testing123",1,4)) 2nd resolved set of brackets for substr = set ui_label myUI myLbl text "test"

Appendix A – Preset Logic continued...

Within presets the following logic commands can be used:

- **instr()** Boolean use returns as false when **<string>** does not contain **<searchString>** and returns true when **<string>** does contain **<searchString>**.
Integer use returns as 0 when **<string>** does not contain **<searchString>** and returns the start position of **<searchString>** within **<string>** from 1.
Optional use of **<offset>** can be used to add or subtract from the resulting index.
- **instrev()** Boolean use returns as false when **<string>** does not contain **<searchString>** and returns true when **<string>** does contain **<searchString>**.
Integer use returns as 0 when **<string>** does not contain **<searchString>** and returns the end position of **<searchString>** within **<string>** from 1.
Optional use of **<offset>** can be used to add or subtract from the resulting index.
- **replace()** Replaces a string with another string.
- **strlen()** Returns the length of the specified **<string>**.
- **substr()** Will extract a string from a string using a starting index and length.
- **substring()** Will extract a string from a string using a starting and ending index.
- **trim()** Removes any hex bytes after the last ASCII character, like terminators **<cr>** from the end of the string.
- **val()** Extract a number from a string.
- **dec()** Decrement the integer value of **<variable>** by **<optional_amount>**, otherwise 1 used as default.
- **inc()** Increment the integer value of **<variable>** by **<optional_amount>**, otherwise 1 used as default.
- **loop while()** Used to repeat a script block.
- **select case()** Used in place of multiple if / elseif statements.
- **return()** Used to override the return status of a preset and optionally terminate the presets execution.
- **suspend()** Used to delay the operation of a command while not delaying the presets execution.

Appendix A – Preset Logic continued...

Within presets the following logical operators are available:

- **&** Used to append strings together.
- **&&** Used as a logical *And/Also* for multiple boolean conditional statements.
- **||** Used as a logical *Or* for multiple boolean conditional statements.
- **==** (equal to) Used to compare numbers or strings that equal.
- **!=** (not equal to) Used to compare numbers or strings that do not equal.
- **<** (less than) Used to compare numbers.
- **>** (greater than) Used to compare numbers.
- **<=** (less than or equal to) Used to compare numbers.
- **>=** (greater than or equal to) Used to compare numbers.
- **not** Used to invert an if statement boolean result.

Within a UI preset the following system read only variables are available:

- **<<ui_name>>** contains the UI name as a string.
- **<<button_name>>** contains the active button name as a string.
- **<<button_position>>** contains the active button position as a string.
- **<<slider_name>>** contains the active slider name as a string.
- **<<slider_value>>** contains the active slider value as an integer.
- **<<encoder_name>>** contains the active Encoder button assigned device name as string.
- **<<decoder_name>>** contains the active Decoder button assigned device name as string.
- **<<pin_code>>** contains the entered pin code as an integer.
- **<<send_response>>** contains a send command response string.

Appendix A – Preset Logic continued...

substr() with instr() example

Combining **substr()** with **instr()** to extract a value from a return string as follows:

```
(substr(<string>,<startIndex>,<optional_numOfCharacters>))  
(instr(<string>,<searchString>,<optional_offset>))  
  
set var LevelTmp (trim(send tcp 10.1.1.10 4998 "get_LED_LIGHTING,1:3\x0D" reply))  
LevelTmp = LED_LIGHTING,1:3,25,50  
  
set var LevelTmp (substr((get var LevelTmp),18))  
LevelTmp = 25,50  
  
set var LevelTmp (substr((get var LevelTmp),0,(instr((get var LevelTmp),",",-1))))  
LevelTmp = 25  
  
set ui_slider myUI mySldr value (get var LevelTmp)  
  
if ((get var LevelTmp) > 0) {  
    set ui_slider myUI mySldr state enabled  
    set var LightLevel (get var LevelTmp)  
}  
  
set var LevelTmp [delete]
```

Appendix A – Preset logic continued...

Command results

The following **get** commands will return a **string** value that can be used with:

- get var
 - get ui
 - get ui_button (state | position)
 - get ui_slider (state)
 - get events (state)
-
- == (equal to)
 - != (not equal to)

Example 1:

```
if ((get var myVar) == "<string>") {  
    <action>  
} else {  
    <action>  
}  
  
select case ((get var myVar)) {  
    case "<string>"  
        <action>  
    case else  
        <action>  
}
```

Example 2:

```
if ((get var myVar) != "<string>") {  
    <action>  
} else {  
    <action>  
}
```

Example 3:

```
if (get ui myUI == "disabled") {  
    <action>  
}  
  
select case ((get ui myUI)) {  
    case "disabled"  
        <action>  
    }  
}
```

Example 4:

```
if (get ui_button myUI myBtn position == "down") {  
    <action>  
}  
  
if (get ui_button myUI myBtn state == "enabled") {  
    <action>  
}
```

Example 5:

```
if (get ui_slider myUI mySldr state == "enabled") {  
    <action>  
}
```

Example 6:

```
if (get events myEvent state != "enabled") {  
    <action>  
}
```

Appendix A – Preset logic continued...

Command results continued...

The following **get** commands will return an **integer** value that can be used with:

- get var
- get indicator (value)
- get slider (value)

- == (equal to)
- != (not equal to)
- < (less than)
- > (greater than)
- <= (less than or equal to)
- >= (greater than or equal to)

Example 1:

```
if ((get var myVar) == 100) {
  <action>
} else {
  <action>
}

select case ((get var myVar)) {
  case "100"
    <action>
  case else
    <action>
}
```

Example 2:

```
if ((get var myVar) != 100) {
  <action>
} else {
  <action>
}
```

Example 3:

```
if ((get ui_indicator myUI mySldr value) == 50) {
  <action>
} else {
  <action>
}

select case ((get ui_indicator myUI mySldr value)) {
  case "50"
    <action>
  case else
    <action>
}
```

Example 4:

```
if ((get ui_indicator myUI myInd value) < 100) {
  <action>
} else {
  <action>
}
```

Example 5:

```
if ((get ui_slider myUI mySldr value) > 100) {
  <action>
} else {
  <action>
}
```

Example 6:

```
if (((get var myVar) <= 100) && ((get var myVar) >= 50)){
  <action>
} else {
  <action>
}
```


Appendix A – Preset logic continued...

Command results continued...

The following **get** command will return a **Boolean** value that can be used with:

- get var
- not

Example 1:

```
if (get var myVar) {  
    <action>  
}  
elseif (get var myVar2) {  
    <action>  
}  
  
select case ((get var myVar)) {  
    case "true"  
        <action>  
    case else  
        select case ((get var myVar2)) {  
            case "true"  
                <action>  
        }  
}
```

Example 2:

```
if not (get var myVar) {  
    <action>  
}  
  
select case ((get var myVar)) {  
    case "false"  
        <action>  
    }  
}
```

Appendix A – Preset logic continued...

Command results continued...

The following **send** commands will return a **string** value that can be used with:

- send tcp <feedback> = reply
 - send udp <feedback> = reply
 - send telnet <feedback> = reply
 - send gc > (port 4999 / 5000) <feedback> = reply
 - send gc > (port 4998) command get
-
- == (equal to)
 - != (not equal to)

Example 1:

```
if (send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" reply == "state,1:1,1\x0D") {  
    <action>  
} else {  
    <action>  
}  
  
if ((trim(send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" reply) == "state,1:1,1") {  
    <action>  
} else {  
    <action>  
}  
  
select case ((send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" reply)) {  
    case "state,1:1,1\x0D"  
        <action>  
    case else  
        <action>  
}
```

Example 2:

```
if (send gc 169.254.1.70 4999 "My String" reply == "This String") {  
    <action>  
}
```

Example 3:

```
if (send gc 169.254.1.70 4998 getstate,1:1 == "state,1:1,1\x0D") {  
    <action>  
}  
  
if (trim(send gc 169.254.1.70 4998 getstate,1:1) == "state,1:1,1") {  
    <action>  
}
```

Example 4:

```
if (send udp 255.255.255.255 10000 "My String" reply == "This String") {  
    <action>  
}
```

Example 5:

```
if (send telnet 10.1.1.10 23 username "Username:" "MyLogin" password "Password:" "MyPassword" ...  
"My String" reply == "This String") {  
    <action>  
}
```


Appendix A – Preset logic continued...

Command results continued...

The following **send** commands will return a **Boolean** value that can be used with:

- send tcp <feedback> = none, equals or contains
 - send udp <feedback> = none, equals or contains
 - send telnet <feedback> = none, equals or contains
 - send gc > (port 4999 / 5000) <feedback> = none, equals or contains
 - send gc > (port 4998)
-
- not

Example 1:

```
if (send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" contains "setstate,1:1,1") {  
    <action>  
} else {  
    <action>  
}  
  
if (send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D") {  
    <action>  
} else {  
    <action>  
}  
  
select case ((send tcp 10.1.1.10 4998 "setstate,1:1,1\x0D" contains "setstate,1:1,1")) {  
    case "true"  
        <action>  
    case else  
        <action>  
}
```

Example 2:

```
if not (send gc 169.254.1.70 4998 setstate,1:1,1) {  
    <action>  
}
```

Example 3:

```
if (send udp 255.255.255.255 10000 "My String" equals "This String") {  
    <action>  
} else {  
    <action>  
}
```

Example 4:

```
if (send telnet 10.1.1.10 23 username "Username:" "MyLogin" password "Password:" "MyPassword" ...  
"My String" contains "This String") {  
    <action>  
} else {  
    <action>  
}
```

Appendix A – Preset logic continued...

Preset results

Preset results determine the button glow as green on success and red on error. Depending on the content of the preset can also determine the default return of a preset. For instance a send tcp command on its own will return true for success or false for error depending on result of the send tcp command itself.

The following preset will return success if a TCP connection was successful or false if unsuccessful:

```
send tcp 10.1.1.10 1234 "send this"
```

The following preset will return success if a TCP connection was successful and the return string contains "this" or error if unsuccessful:

```
send tcp 10.1.1.10 1234 "send this" contains "this"
```

The following preset will return success even if a TCP connection was unsuccessful or the return string was no correct:

```
if (send tcp 10.1.1.10 1234 "send this" contains "this") {  
    <action>  
}
```

The reason this is because the result of the send tcp command is used for the if statement, not the preset return.

To control the return of a preset the following return expressions can be applied:

- `return(true)` Override the result as success with green button glow
- `return(false)` Override the result as error with red button glow
- `return(null)` Override the result as success with no button glow

- `return(true,false)` Override the result as success with green button glow
- `return(false,false)` Override the result as error with red button glow
- `return(null,false)` Override the result as success with no button glow

- `return(true,true)` Terminate preset at current location as success with green button glow
- `return(false,true)` Terminate preset at current location as error with red button glow
- `return(null,true)` Terminate preset at current location as success with no button glow

Appendix A – Preset logic continued...

Preset result examples of default results

Presets returning as error are subject to preset error notification emails (refer to Notifications) and logging the preset error.

- The following preset will return **success** with a **green** button glow:

```
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `send tcp 10.1.1.10 1234 "command" contains "result"` then **terminate** returning **error** with a **red** button glow when the reply does **not** contain "result":

```
send tcp 10.1.1.10 1234 "command" contains "result"
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute then return **success** with a **green** button glow when the reply does contain "result":

```
send tcp 10.1.1.10 1234 "command" contains "result"
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
    <false_returning_preset_command>
}
```

- The following preset will execute then return **success** with a **green** button glow when the reply does **not** contain "result":

```
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
    <true_returning_preset_command>
}
```

Appendix A – Preset logic continued...

Preset result examples using return(true)

Presets returning as error are subject to preset error notification emails (refer to Notifications) either logging the preset error or return command use.

- The following preset will execute then return **success** with a **green** button glow:

```
return(true)
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **success** with a **green** button glow:

```
return(true)
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
return(true)
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true)
}
<false_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **success** with a **green** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true)
}
<false_returning_preset_command>
```

- The following preset will execute then return **success** with a **green** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true)
}
<true_returning_preset_command>
```

- The following preset will execute then return **success** with a **green** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true)
}
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<false_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true)
}
<true_returning_preset_command>
```

Appendix A – Preset logic continued...

Preset result examples using return(false)

Presets returning as error are subject to preset error notification emails (refer to Notifications) either logging the preset error or return command use.

- The following preset will execute then return **error** with a **red** button glow:

```
return(false)
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
return(false)
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
return(false)
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false)
}
<false_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false)
}
<false_returning_preset_command>
```

- The following preset will execute then return **success** with a **green** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false)
}
<true_returning_preset_command>
```

- The following preset will execute then return **error** with a **red** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false)
}
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<false_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false)
}
<true_returning_preset_command>
```

Appendix A – Preset logic continued...

Preset result examples using return(null)

Presets returning as error are subject to preset error notification emails (refer to Notifications) either logging the preset error or return command use.

- The following preset will execute then return **success** with a no button glow:

```
return(null)
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **true** with a no button glow:

```
return(null)
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
return(null)
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null)
}
<false_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **success** with no button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null)
}
<false_returning_preset_command>
```

- The following preset will execute then return **success** with no button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null)
}
<true_returning_preset_command>
```

- The following preset will execute then return **success** with a **green** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null)
}
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<false_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null)
}
<true_returning_preset_command>
```

Appendix A – Preset logic continued...

Preset result examples using return(true,true)

Presets returning as error are subject to preset error notification emails (refer to Notifications) either logging the preset error or return command use.

- The following preset will **terminate** then return **success** with a **green** button glow:

```
return(true,true)
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will **terminate** then return **success** with a **green** button glow:

```
return(true,true)
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
return(true,true)
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true,true)
}
<false_returning_preset_command>
```

- The following preset will **terminate** returning **success** with a **green** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true,true)
}
<false_returning_preset_command>
```

- The following preset will execute then return **success** with a **green** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true,true)
}
<true_returning_preset_command>
```

- The following preset will **terminate** returning **success** with a **green** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true,true)
}
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<false_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(true,true)
}
<true_returning_preset_command>
```


Appendix A – Preset logic continued...

Preset result examples using return(false,true)

Presets returning as error are subject to preset error notification emails (refer to Notifications) either logging the preset error or return command use.

- The following preset will **terminate** then return **error** with a **red** button glow:

```
return(false,true)
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will **terminate** then return **error** with a **red** button glow:

```
return(false,true)
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
return(false,true)
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false,true)
}
<false_returning_preset_command>
```

- The following preset will **terminate** returning **error** with a **red** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false,true)
}
<false_returning_preset_command>
```

- The following preset will execute then return **true** with a **green** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false,true)
}
<true_returning_preset_command>
```

- The following preset will execute then return **error** with a **red** button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false,true)
}
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<false_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(false,true)
}
<true_returning_preset_command>
```

Appendix A – Preset logic continued...

Preset result examples using return(null,true)

Presets returning as error are subject to preset error notification emails (refer to Notifications) either logging the preset error or return command use.

- The following preset will **terminate** returning **success** with a no button glow:

```
return(null,true)
<true_returning_preset_command>
<true_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will **terminate** returning **success** with a no button glow:

```
return(null,true)
<true_returning_preset_command>
<false_returning_preset_command>
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<true_returning_preset_command>
<false_returning_preset_command>
return(null,true)
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null,true)
}
<false_returning_preset_command>
```

- The following preset will **terminate** returning **success** with no button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null,true)
}
<false_returning_preset_command>
```

- The following preset will execute then return **success** with no button glow when the reply does contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null,true)
}
<true_returning_preset_command>
```

- The following preset will **terminate** returning **success** with no button glow when the reply does **not** contain "result":

```
<true_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null,true)
}
<true_returning_preset_command>
```

- The following preset will execute `<false_returning_preset_command>` then **terminate** returning **error** with a **red** button glow:

```
<false_returning_preset_command>
if not (send tcp 10.1.1.10 1234 "command" contains "result") {
  <true_returning_preset_command>
  return(null,true)
}
<true_returning_preset_command>
```


Appendix A – Preset logic continued...

Logic Examples

Using a variable as string to store a send tcp/udp/telnet command result string:

```
set var myVar (send tcp 10.1.1.10 1234 "{status}\x0D" reply)

if ((get var myVar) == "option1") {
    <action>
} elseif ((get var myVar) == "option2") {
    <action>
} elseif ((get var myVar) == "option3") {
    <action>
} else {
    <action>
}

select case ((get var myVar)) {
    case "option1"
        <action>
    case "option2"
        <action>
    case "option3"
        <action>
    case else
        <action>
}

set ui_label myUI myLbl text (get var myVar)
```

Using a variable as Boolean to set a button state:

```
set var myVar false

if ((get var myVar)) {
    set ui_button myUI myBtn position up
} else {
    set ui_button myUI myBtn position down
}

select case ((get var myVar)) {
    case "true"
        set ui_button myUI myBtn position up
    case else
        set ui_button myUI myBtn position down
}
```

Check if a variable exists:

```
if ((get var myVar)) {
    //variable exists
} else {
    //variable does not exist or boolean false
}
```

Appendix A – Preset logic continued...

Logic Examples continued...

Using volume buttons:

```
if ((<<button_name>> == "volUP") && ((get var myVol) < 100)) {
    inc(myVol)
} elseif ((<<button_name>> == "volDOWN") && ((get var myVol) > 0)) {
    dec(myVol)
}

select case (<<button_name>>) {
    case "volUP"
        if ((get var myVol) < 100)) {
            inc(myVol)
        }
    case "volDOWN"
        if ((get var myVol) > 0)) {
            dec(myVol)
        }
}

send tcp 10.1.1.10 1234 ("vol=" & (get var myVol) & "\x0D")
```

TV channel status example using if / elseif:

```
set var myChannel (trim(send tcp 10.1.1.10 1234 "ch?\x0D" reply))
if (get var myChannel == "ch:1") {
    set ui_button <<ui_name>> ch1 position down
} elseif (get var myChannel == "ch:2") {
    set ui_button <<ui_name>> ch1 position down
} elseif (get var myChannel == "ch:2") {
    set ui_button <<ui_name>> ch2 position down
} elseif (get var myChannel == "ch:3") {
    set ui_button <<ui_name>> ch3 position down
} elseif (get var myChannel == "ch:4") {
    set ui_button <<ui_name>> ch4 position down
} elseif (get var myChannel == "ch:5") {
    set ui_button <<ui_name>> ch5 position down
} elseif (get var myChannel == "ch:6") {
    set ui_button <<ui_name>> ch6 position down
} elseif (get var myChannel == "ch:7") {
    set ui_button <<ui_name>> ch7 position down
} elseif (get var myChannel == "ch:8") {
    set ui_button <<ui_name>> ch8 position down
} elseif (get var myChannel == "ch:9") {
    set ui_button <<ui_name>> ch9 position down
} elseif (get var myChannel == "ch:10") {
    set ui_button <<ui_name>> ch10 position down
} else {
    set ui_button <<ui_name>> TVchannels position up
}
```

Appendix A – Preset logic continued...

Logic Examples continued...

TV channel status example using select case:

```
set var myChannel (trim(send tcp 10.1.1.10 1234 "ch?\x0D" reply))
select case (get var myChannel) {
  case "ch:1"
    set ui_button <<ui_name>> ch1 position down
  case "ch:2"
    set ui_button <<ui_name>> ch2 position down
  case "ch:3"
    set ui_button <<ui_name>> ch3 position down
  case "ch:4"
    set ui_button <<ui_name>> ch4 position down
  case "ch:5"
    set ui_button <<ui_name>> ch5 position down
  case "ch:6"
    set ui_button <<ui_name>> ch6 position down
  case "ch:7"
    set ui_button <<ui_name>> ch7 position down
  case "ch:8"
    set ui_button <<ui_name>> ch8 position down
  case "ch:9") {
    set ui_button <<ui_name>> ch9 position down
  case "ch:10") {
    set ui_button <<ui_name>> ch10 position down
  case else {
    set ui_button <<ui_name>> TVchannels position up
}
```

Appendix A – Preset logic continued...

Logic Examples continued...

TV channel status example using element name from variable:

```
set var myChannel (trim(send tcp 10.1.1.10 1234 "ch?\x0D" reply))
set var myChannel (replace((get var myChannel),":",""))
if (instr((get var myChannel),"ch")) {
    set ui_button <<ui_name>> (get var myChannel) position down
} else {
    set ui_button <<ui_name>> TVchannels position up
}
```

Setting status from binary data example:

```
set var myPwrStatus (trim(send tcp 10.1.1.10 1234 "pwr?\x0D" reply))
myPwrStatus = "11000"

if ((substr((get var myPwrStatus),1,1)) == "1") {
    set ui_button <<ui_name>> btn_pwr_tv position down
} else {
    set ui_button <<ui_name>> btn_pwr_tv position up
}

if ((substr((get var myPwrStatus),2,1)) == "1") {
    set ui_button <<ui_name>> btn_pwr_amp position down
} else {
    set ui_button <<ui_name>> btn_pwr_amp position up
}

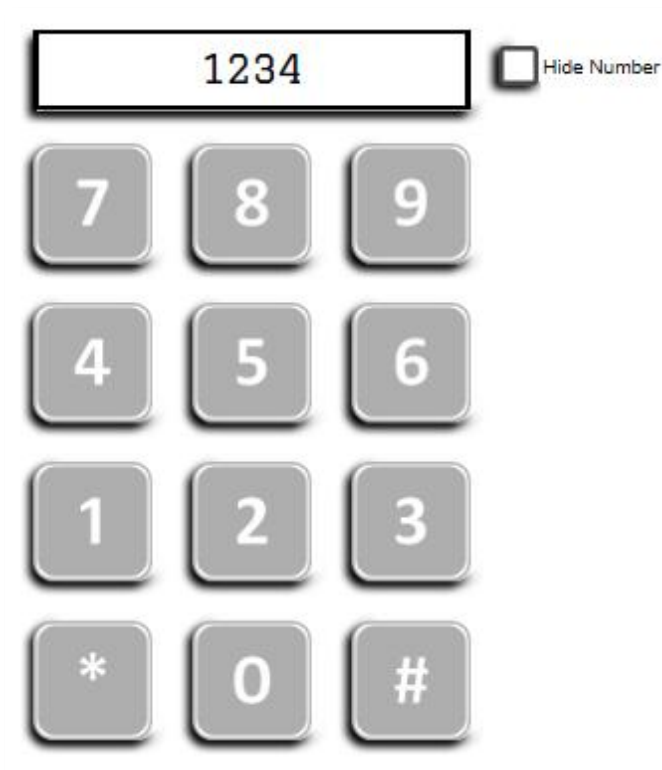
if ((substr((get var myPwrStatus),3,1)) == "1") {
    set ui_button <<ui_name>> btn_pwr_sub position down
} else {
    set ui_button <<ui_name>> btn_pwr_sub position up
}

if ((substr((get var myPwrStatus),4,1)) == "1") {
    set ui_button <<ui_name>> btn_pwr_bluray position down
} else {
    set ui_button <<ui_name>> btn_pwr_bluray position up
}

if ((substr((get var myPwrStatus),5,1)) == "1") {
    set ui_button <<ui_name>> btn_pwr_dvd position down
} else {
    set ui_button <<ui_name>> btn_pwr_dvd position up
}
```

Appendix B – UI Snippets

Login Codes – Refer UI project: demo_ui_login_codes



The image shows a UI snippet for a login code input. It consists of a text box at the top containing the digits '1234'. To the right of the text box is a checkbox labeled 'Hide Number'. Below the text box is a numeric keypad with four rows of three buttons each. The buttons are labeled with the following characters: Row 1: 7, 8, 9; Row 2: 4, 5, 6; Row 3: 1, 2, 3; Row 4: *, 0, #.

When placed on the UI home page this custom login snippet will allow different pin codes to open specific UI pages. This example uses a 4 digit pin code but any length code can be used. Different options for invalid codes can be customised to your requirements.

Preset **demo_ui_login_codes** is used for all functionality.

COMMAND GUIDE V1.3.8.0

Appendix B – UI Snippets
continued...

Login Codes continued...

```
// demo_ui_login_codes V0.2.0

return(null) // prevent button glow

// check is onload called preset and reset pin code
if (<<button_name>> == "button_name"){ // onload called

    set var pinCode "" // clear pin code
    set ui_label <<ui_name>> lbl_pin text "" // clear display
    return(null,true) // return finished
}

// check which button was pressed

if (<<button_name>> == "btn_chkbox") { // check if hide pressed

    if (<<button_position>> == "down") {

        if ((strlen((get var pinCode))) > 0){ // check if pin code to hide
                                                // hide pin code
            set ui_label <<ui_name>> lbl_pin text ((substr("****",0,(strlen((get var pinCode))))))
        }

    } else { // btn_chkbox up

        set ui_label <<ui_name>> lbl_pin text (get var pinCode) // display pin code
    }

} elseif ((val(<<button_name>>)) > 9) { // buttons * or # clear pin code

    set var pinCode "" // clear the pin code
    set ui_label <<ui_name>> lbl_pin text "" // clear display

} else { // button 0 - 9

    set var pinCode ((get var pinCode) & (val(<<button_name>>))) // add entered number to pin code

    if (get ui_button <<ui_name>> btn_chkbox position == "down") { // check if hide pressed

        set ui_label <<ui_name>> lbl_pin text ...
        ((substring("****",0,(strlen((get var pinCode))-1))) ...
        & (substr((get var pinCode),(strlen((get var pinCode)))))) // hide leading digits

        suspend((set ui_label <<ui_name>> lbl_pin text ((substr("****",0,...
        (strlen((get var pinCode)))))),0.5) // hide the digits after 500ms

    } else { // display number selected

        set ui_label <<ui_name>> lbl_pin text (get var pinCode) // display pin code
    }

}

...
```


Appendix B – UI Snippets continued...

Login Codes continued...

```
select case (get var pinCode) {                                     // check the pin code

    case "1111"
        set var pinCode ""                                         // clear the pin code
        preset delay 500                                           // delay to display pin
        set ui_page <<ui_name>> "page1"                             // change page
        set ui_label <<ui_name>> lbl_pin text ""                   // clear display
        set ui_button <<ui_name>> btn_chkbox position up           // set hide button up

    case "2222"
        set var pinCode ""                                         // clear the pin code
        preset delay 500                                           // delay to display pin
        set ui_page <<ui_name>> "page2"                             // change page
        set ui_label <<ui_name>> lbl_pin text ""                   // clear display
        set ui_button <<ui_name>> btn_chkbox position up           // set hide button up

    case else

        // optionally do something when wrong code entered
        if ((strlen((get var pinCode))) == 4) {                   // change 4 to length of code

            preset delay 500                                       // delay 500ms
            set ui_label <<ui_name>> lbl_pin color FF0000          // change display text to red
            set ui_label <<ui_name>> lbl_pin text (get var pinCode) // display entered code

            set var pinCode ""                                     // reset the pin code

            // at this point we want to disable all the buttons
            // individually is possible but not as a group of momentary buttons, YET!
            //set ui_button <<ui_name>> pinGroup state disabled // disable group
            set ui_button <<ui_name>> numpad_0 state disabled        // disable button numpad_0
            ...                                                    // disable buttons numpad_1-10
            set ui_button <<ui_name>> numpad_11 state disabled       // disable button numpad_11

            set ui_button <<ui_name>> btn_chkbox state disabled // disable check box

            preset delay 10000                                     // delay for 10 seconds

            //set ui_button <<ui_name>> pinGroup state enabled // enable group
            set ui_button <<ui_name>> numpad_0 state enabled        // enable button numpad_0
            ...                                                    // enable buttons numpad_1-10
            set ui_button <<ui_name>> numpad_11 state enabled       // enable button numpad_11

            set ui_button <<ui_name>> btn_chkbox state enabled // enable check box
            set ui_label <<ui_name>> lbl_pin text ""               // clear display
            set ui_label <<ui_name>> lbl_pin color 000000          // change display text to black

        }

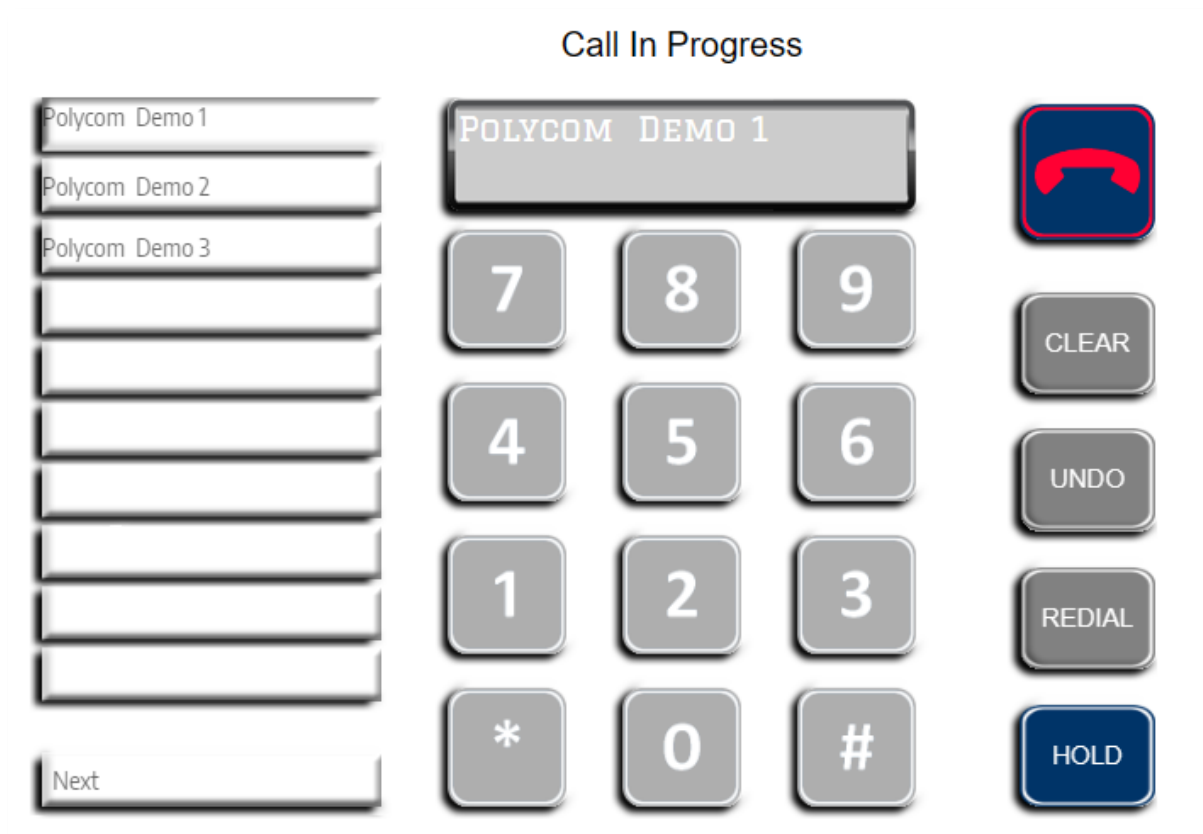
        // or optionally do something when too many digits entered
        //if ((strlen((get var pinCode))) > 4) {                   // change 4 to length of code
        //    set var pinCode ""                                     // clear the pin code
        //    return(false)                                         // glow button red
        //}

    }

}
```


Appendix B – UI Snippets continued...

Phone Dialer – Refer UI project: demo_ui_phone_dialer



Polycom commands have been used for this example with address book and command feedback.

Preset **demo_ui_phone_dialer** is used for all button functionality while **demo_ui_phone_addrbook** preset is used to obtain the address book details.

Note: If a Telnet login or constant TCP connection is required to the phone system then simply substitute any **send tcp** command with **sent telnet** and include any login details.

Appendix B – UI Snippets continued...

Phone Dialer continued...

```
// demo_ui_phone_dialer V0.3.0

return(null) // prevent button glow

set ui_label <<ui_name>> lbl_ph_status text "" // clear status

// check which button is pressed

if ((instr( <<button_name>> ,"addrbookbtn")) { // addrbook button pressed

    set var phNum (get ui_button <<ui_name>> <<button_name>> text up) // store phNum
    set ui_button <<ui_name>> btn_clear state enabled // enable clear button

    if (get ui_button <<ui_name>> btn_call position == "down") { // check if btn_call down

        if (send tcp (get var dialerIP) 23 ("dial addressbook "" ...
            & (get ui_button <<ui_name>> <<button_name>> text up) & ""\x0D\x0A") { // send command

            set ui_label <<ui_name>> lbl_number text ...
                (get ui_button <<ui_name>> <<button_name>> text up) // display addrbook entry

            set ui_button <<ui_name>> btn_undo state disabled // disable undo button
            set ui_button <<ui_name>> btn_redial state disabled // disable redial button
            set ui_button <<ui_name>> btn_call position down // call button down
            set ui_button <<ui_name>> btn_hold state enabled //enable hold button
            set ui_button <<ui_name>> addrbookbtngroup state disabled // disable addrbook button
            set ui_button <<ui_name>> addrbookbtn_next state disabled // disable next button
            set ui_label <<ui_name>> lbl_status text "Call In Progress"// show status

            return(null,true) // return no button glow

        } else { // error sending command

            set ui_label <<ui_name>> lbl_ph_status text "Error Dialing Number" // display error

            return(false,true) // return glow button red

        }

    } else { // call button up

        set ui_button <<ui_name>> btn_clear state enabled // enable clear button

    }

    ...
}
```

Appendix B – UI Snippets continued...

Phone Dialer continued...

```

} elseif ((instr(<<button_name>>,"numpad"))) {                                // number pad button

    if (get ui_button <<ui_name>> btn_call position == "up") {                // on hook

        set ui_button <<ui_name>> btn_clear state enabled                    // enable clear button
        set ui_button <<ui_name>> btn_undo state enabled                      // enable undo button

        if (<<button_name>> == "numpad_star") {                                // check if * button

            set var phNum ((get var phNum) & "*")                            // add * to ph number

        } elseif (<<button_name>> == "numpad_hash") {                          // check if # button

            set var phNum ((get var phNum) & "#")                            // add # to ph number

        } else {                                                            // button 0 - 9

            set var phNum ((get var phNum) & (val(<<button_name>>)))          // add to digit ph number

        }

    } else {                                                                // off hook

        set ui_button <<ui_name>> btn_hold state enabled                    // enable hold button

        if (<<button_name>> == "numpad_star") {                                // check if * button

            set var phTemp "*"                                                // store digit

        } elseif (<<button_name>> == "numpad_hash") {                          // check if # button

            set var phTemp "#"                                                // store digit

        } else {                                                            // button 0 - 9

            set var phTemp (val(<<button_name>>))                              // store digit

        }                                                                    // send entered digit

        if (send tcp (get var dialerIP) 23 ...
            ("gendial " & (get var phTemp) & "\x0D\x0A") contains "gendial") {

            set var phNum ((get var phNum) & (get var phTemp))                // append digit to ph num
            set ui_label <<ui_name>> lbl_number color FFFFFFFF                // change display to white

        } else {                                                            // error sending digit

            return(false)                                                    // glow button red

        }

        set var phTemp ""                                                    // clear stored digit

    }

} else {                                                                    // ! addrbookbtn or numpad

    ...

```

COMMAND GUIDE V1.3.8.0

Appendix B – UI Snippets
continued...

Phone Dialer continued...

```
select case (<<button_name>>) { // select pressed button

    case "btn_call" // call button pressed

        if (<<button_position>> == "down") { // dial number

            set ui_label <<ui_name>> lbl_status text "Call In Progress" // show status
            set ui_button <<ui_name>> btn_redial state disabled // disable redial button

            if ((strlen((get var phNum))) > 0) { // chk entered ph number

                set var redialNum (get var phNum) // store number dialed

                if (send tcp (get var dialerIP) 23 ... // send command
                    ("dial auto "" & (get var phNum) & ""\x0D\x0A") contains "dial auto") {

                    set ui_button <<ui_name>> btn_clear state disabled // disable clear button
                    set ui_button <<ui_name>> btn_undo state disabled // disable undo button
                    set ui_button <<ui_name>> btn_hold state enabled // disable hold button
                    set ui_button <<ui_name>> addrbookbtngroup state disabled // disable addrbook
                    set ui_button <<ui_name>> addrbookbtn_next state disabled // disable next btn
                    set ui_label <<ui_name>> lbl_number color FFFFFFFF // change display to white

                } else { // error sending command

                    set ui_button <<ui_name>> <<button_name>> position up // revert btn position
                    return(false) // glow button red

                }

            } else { // no entered ph number

                set var redialNum "" // clear redial number

            }

        } else { // hang up

            set ui_button <<ui_name>> addrbookbtngroup position up // addrbook buttons up

        }

    ...
```

Appendix B – UI Snippets continued...

Phone Dialer continued...

```
// next the phonebook buttons need to be enabled that contain text which ideally would be executed
// in a loop which currently is not supported so must be repeated to include the number of
// buttons used, in this case 10

// future method

//set var x 0 // reset counter

//loop while (x < 10) { // command loop

//    inc(x) // increment counter

//    set var myBtn ("addrbookbtn_" & (get var x)) // create and store btn id

//    if ((strlen((get ui_button <<ui_name>> (get var myBtn) text up))) > 0) { // txt
//        set ui_button <<ui_name>> (get var myBtn) state enabled // enable button
//    }

//}

if ((strlen((get ui_button <<ui_name>> addrbookbtn_1 text up))) > 0) { // check text
    set ui_button <<ui_name>> addrbookbtn_1 state enabled // enable button if text
}

... // repeated x8

if ((strlen((get ui_button <<ui_name>> addrbookbtn_10 text up))) > 0) { // check text
    set ui_button <<ui_name>> addrbookbtn_10 state enabled // enable button if text
}

if ((get var addrbooknextenabled)) { // check next button state
    set ui_button <<ui_name>> addrbookbtn_next state enabled // enable next button
} else {
    set ui_button <<ui_name>> addrbookbtn_next state disabled // disable next button
}

...
```

Appendix B – UI Snippets continued...

Phone Dialer continued...

```

        if ((strlen((get var phNum))) > 0) {                                     // if ph number available

            // send the command with feedback and deal with success and error command returns

            if (send tcp (get var dialerIP) 23 ...
                "hangup all\x0D\x0A" contains "hanging up all") {

                if ((strlen((get var redialNum))) > 0) {                         // chk if redial available
                    set ui_button <<ui_name>> btn_redial state enabled // enable redial btn
                }
                set ui_button <<ui_name>> btn_hold state disabled // disable hold button
                set ui_button <<ui_name>> btn_hold position up // hold button up
                set ui_label <<ui_name>> lbl_number color 000000 // change display to black
                set ui_label <<ui_name>> lbl_status text "" // clear status

                set var phNum "" // clear ph number

            } else { // error sending command

                set ui_button <<ui_name>> <<button_name>> position down // revert position
                return(false) // glow button red

            }

        } else { // no ph number available

            set ui_label <<ui_name>> lbl_status text "" // clear status

        }

    }

case "btn_clear" // clear button pressed

    if ((get var phNum) == (get var redialNum)) { // ph number = redial num

        set ui_button <<ui_name>> btn_redial state disabled // disable redial button
        set var redialNum "" // clear redial number

    }

    set var phNum "" // clear ph number
    set ui_button <<ui_name>> btn_undo state disabled // disable undo button
    set ui_button <<ui_name>> <<button_name>> state disabled // disable clear button
    set ui_button <<ui_name>> addrbookbtngroup position up // addrbook buttons up

case "btn_undo" // undo button pressed

    set var phNum (substring((get var phNum),0,-1)) // remove last digit

    if ((strlen((get var phNum))) == 0) { // chk if ph num cleared

        set ui_button <<ui_name>> btn_clear state disabled // disable clear button
        set ui_button <<ui_name>> <<button_name>> state disabled // disable undo button

    }

...

```


Appendix B – UI Snippets continued...

Phone Dialer continued...

```
case "btn_redial" // redial button pressed

set ui_button <<ui_name>> btn_clear state enabled // disable clear button
set ui_button <<ui_name>> btn_undo state disabled // disable undo button
set ui_button <<ui_name>> <<button_name>> state disabled // disable redial button

set var phNum (get var redialNum) // set phone number

case "btn_hold" // hold button pressed

if (<<button_position>> == "down") { // check if button down

    // send the command with feedback and deal with success and error command returns

    if (send tcp (get var dialerIP) 23 ...
        "mute near on\x0D\x0A" contains "mute near on") { // success

        set ui_label <<ui_name>> lbl_status text "Call On Hold" // update status

    } else { // error sending command

        set ui_button <<ui_name>> <<button_name>> position up // revert button position
        return(false) // glow button red

    }

} else { // button up

    // send the command with feedback and deal with success and error command returns

    if (send tcp (get var dialerIP) 23 ...
        "mute near off\x0D\x0A" contains "mute near off") { // success

        set ui_label <<ui_name>> lbl_status text "Call In Progress" // update status

    } else { // error sending command

        set ui_button <<ui_name>> <<button_name>> position down // revert button position
        return(false) // glow button red

    }

}

} // end select case

set ui_label <<ui_name>> lbl_number text (get var phNum) // display ph number

// the end
```

Appendix B – UI Snippets continued...

Phone Dialer continued...

The following string is an address book reply example:

```
addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0Aaddrbook 1.
"Polycom Demo 2" sip_spd:384 sip_num:polycomg7500@polycom.com\x0D\x0Aaddrbook 2. "Polycom Demo 3"
phone_num:1.512.5121212\x0D\x0Aaddrbook all done\x0D\x0A
```

The main function of this script is to obtain the phone Directory data and process the names in between "quotation marks" to display, and use for the dialing command.

```
addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0A
```

- Sequentially working the reply the script obtains the first instance of a " quotation mark as **indexStart**.
addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0A
- Then obtains the following " quotation mark instances as **indexEnd**.
addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0A
- Next obtains the name in between the "quotation marks" **indexStart** and **indexEnd**.
addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0A
- Finally removing the text from the string.
~~addrbook 0. "Polycom Demo 1" h323_spd:384 h323_num:192.168.1.101 h323_ext:7878\x0D\x0A~~

This is repeated for the number of Directory buttons used (*in this case 10*) or number of name entries. After processing up to 10 entries and more still remain the Next button is enabled. Pressing the Next button will call this preset and process the reaming or another 10 entries. This will continue until there are no more entries available, at which point the address book reply string is refreshed and the whole process is repeated.

COMMAND GUIDE V1.3.8.0

Appendix B – UI Snippets
continued...

Phone Dialer continued...

```
// demo_ui_phone_addrbook V0.3.0

set var dialerIP "172.30.10.166" // set the IP Address

return(null) // prevent button glow

// do not allow status during a phone call which can be triggered refreshing the browser

if (get ui_button <<ui_name>> btn_call position == "down") {
    return(null,true)
}

// clear the ui

set ui_button <<ui_name>> addrbookbtngroup state disabled // disable addrbook button
set ui_button <<ui_name>> addrbookbtngroup position up // addrbook buttons up
set ui_button <<ui_name>> addrbookbtngroup text both "" // clear addrbook buttons
set ui_label <<ui_name>> lbl_ph_status text "" // clear status text

// check if Next button pressed else obtain new address book data

if not (<<button_name>> == "addrbookbtn_next") { // not next button pressed

    set ui_label <<ui_name>> lbl_ph_status text "Fetching..." // update status
    set ui_button <<ui_name>> addrbookbtn_next state disabled // disable next button
    set var addrbooknextenabled false // reset next button flag

    set var addrbook (send tcp (get var dialerIP) 23 "addrbook all\x0D\x0A" reply) //send command

    if ((instr((get var addrbook),"addrbook all done"))) { // check return

        // 'addrbook', along with 'all done', are specifically being removed to continue
        // using the word 'addrbook' to detect entries

        set var addrbook (replace((get var addrbook),"addrbook all done\x0D\x0A","")) //remove string
        set var addrbookorg (get var addrbook) // store addrbook

    } else { // return not complete

        set var addrbook "" // clear addrbook
        set var addrbookorg "" // clear addrbook copy
        set ui_label <<ui_name>> lbl_ph_status text "Error Address Book" // update status
        return(false,true) // return red button glow

    }

} else { // next button pressed

    // when the Next button is pressed check if further entries are available in var addrbook
    // and if not restore the original data from var addrbookorg
    // the script will continue on with whatever data is in var addrbook

    if not ((instr((get var addrbook),"addrbook"))) { // check for remaining entries
        set var addrbook (get var addrbookorg) // if not reset addrbook data
    }

}

...
```


Appendix B – UI Snippets continued...

Phone Dialer continued...

```
// now the address book data is processed to obtain the names within which
// are then shown on the Directory buttons and used with the dialing command

// ideally the following would be executed in a loop which currently is not supported so must be
// repeated to include the number of Directory buttons used, in this case 10

// future method

//set var x 0 // reset counter

//loop while (x < 10) { // command loop

// if ((instr((get var addrbook),"addrbook")) { // check addrbook entries

// inc(x) // increment counter

// set var indexStart (((instr((get var addrbook),"",1))) // obtain first "
// set var indexEnd ((instr((substr((get var addrbook),(get var indexStart)),"")) // next "

// (inc(indexEnd,(get var indexStart))) // + values > substring()
// dec(indexEnd) // -1 > substring()

// set var myBtn ("addrbookbtn_" & (get var x)) // create and store btn id

// set ui_button <<ui_name>> (get var myBtn) text both ...
// (substring((get var addrbook),(get var indexStart),(get var indexEnd))) // display name

// set ui_button <<ui_name>> (get var myBtn) state enabled // enable addrbook button

// set var addrbook ((substring((get var addrbook),...
// (instr((get var addrbook),"\x0D\x0A",8)))) // remove name from string

// } else { // no more entries

// inc(x,10) // stop looping

// }

//}

...
```

COMMAND GUIDE V1.3.8.0

Appendix B – UI Snippets
continued...

Phone Dialer continued...

```
if ((instr((get var addrbook),"addrbook"))) { // check addrbook entries

    set var indexStart (((instr((get var addrbook),"",1)))) // obtain first "
    set var indexEnd ((instr((substr((get var addrbook),(get var indexStart)),"")))) // next "

    (inc(indexEnd,(get var indexStart))) // + values > substring()
    dec(indexEnd) // -1 > substring()

    set ui_button <<ui_name>> addrbookbtn_1 text both ...
        (substring((get var addrbook),(get var indexStart),(get var indexEnd))) // display name

    set ui_button <<ui_name>> addrbookbtn_1 state enabled // enable addrbook button

    set var addrbook ((substring((get var addrbook),...
        (instr((get var addrbook),"\x0D\x0A",8)))) // remove name from string

    ... // repeated x8

if ((instr((get var addrbook),"addrbook"))) { // check addrbook entries

    set var indexStart (((instr((get var addrbook),"",1)))) // obtain first "
    set var indexEnd ((instr((substr((get var addrbook),(get var indexStart)),"")))) // next "

    (inc(indexEnd,(get var indexStart))) // + values > substring()
    dec(indexEnd) // -1 > substring()

    set ui_button <<ui_name>> addrbookbtn_10 text both ...
        (substring((get var addrbook),(get var indexStart),(get var indexEnd))) // display name

    set ui_button <<ui_name>> addrbookbtn_10 state enabled // enable addrbook button

    set var addrbook ((substring((get var addrbook),...
        (instr((get var addrbook),"\x0D\x0A",8)))) // remove name from string

}
}
}
}
}
}
}
}
}

// after processing the Directory data var addrbook is examined for further
// entries and if available enabling the function of the Next button

if ((instr((get var addrbook),"addrbook"))) { // check if entries remain

    set var addrbooknextenabled true // set next button flag
    set ui_button <<ui_name>> addrbookbtn_next state enabled // enable next button

}

set ui_label <<ui_name>> lbl_ph_status text "" // clear status

// the end
```

Appendix C - How to HTTP request

GET = `http://<controllerURL>/api/command/<ARRANGER_API_COMMAND>/<KEY>`

POST = `http://<controllerURL>/api/command{"cmd": "<ARRANGER_API_COMMAND>", "key": "<KEY>"}`

Example 1: POST - ajax

```
<script language='JavaScript' type='text/javascript'>
  var controllerIP = '10.1.1.10'; *change this to the same IP address as the SDVoE Arranger controller
  var BaseURL = 'http://' + controllerIP + '/api/command';
  var MAXIMUM_WAITING_TIME = 5000; *timeout in milliseconds
  var CheckStatusTimer;
  var key = '123xyz'; *replace this with the generated security key
  var command = '123xyz'; *replace with any Arranger API command

  $.ajax({
    type: 'POST',
    crossDomain: true,
    contentType: 'application/json; charset=utf-8',
    dataType: 'text',
    url: BaseURL,
    data: '{"cmd":"' + command + '", "key":"' + key + '"}',
    timeout: MAXIMUM_WAITING_TIME,
    success: function(data, textStatus, XMLHttpRequest) {
      console.log(data);
    },
    error: function (XMLHttpRequest, textStatus, errorThrown) {
      console.log('ERROR = ' + errorThrown);
    }
  });
</script>
```

Example 2: POST - xhr

```
<script language='JavaScript' type='text/javascript'>
  var controllerIP = '10.1.1.10'; *change this to the same IP address as the SDVoE Arranger controller
  var BaseURL = 'http://' + controllerIP + '/api/command';
  var key = '123xyz'; *replace this with the generated security key
  var command = '123xyz'; *replace with any ARRANGER API command
  var xmlRequest = new XMLHttpRequest();
  xmlRequest.open('POST', BaseURL, true);
  var params = '{"cmd":"' + command + '", "key":"' + key + '"}';
  var MAXIMUM_WAITING_TIME = 5000;
  xmlRequest.onreadystatechange = function () {
    if (this.readyState == 4) {
      clearTimeout(xmlTimer);
      if (this.status == 200) {
        console.log(this.responseText);
      } else {
        console.log('ERROR = ' + this.status + ' ' + this.statusText);
      }
    } else {
      if (this.status != 200) {
        console.log('ERROR = ' + this.status + ' ' + this.statusText);
      }
    }
  };
  xmlRequest.send(params);
  var xmlTimer = setTimeout(function() {
    xmlRequest.abort();
    console.log('ERROR = timeout');
  }, MAXIMUM_WAITING_TIME);
</script>
```

Appendix C - How to HTTP request continued...

Example 3: GET - xhr

```
<script language='JavaScript' type='text/javascript'>
  var controllerIP = '10.1.1.10'; *change this to the same IP address as the SDVoE Arranger controller
  var BaseURL = 'http://' + controllerIP + '/api/command/';
  var key = '123xyz'; *replace this with the generated security key
  var command = '123xyz'; *replace with an ARRANGER API command
  var MAXIMUM_WAITING_TIME = 5000;
  var btn2xhr = new XMLHttpRequest();
  btn2xhr.open('GET', BaseURL + '<command>/' + key);
  btn2xhr.onreadystatechange = function () {
    if (this.readyState == 4) {
      clearTimeout(btn2xhrTimer);
      if (this.status == 200) {
        document.getElementById('Text0').innerHTML = this.responseText;
      } else {
        document.getElementById('Text0').innerHTML = 'ERROR = ' + this.status + ' ' + this.statusText;
      }
    } else {
      if (this.status != 200) {
        document.getElementById('Text0').innerHTML = 'ERROR = ' + this.status + ' ' + this.statusText;
      }
    }
  };
  btn2xhr.send(null);
  var btn2xhrTimer = setTimeout(function() {
    btn2xhr.abort();
    document.getElementById('Text0').innerHTML = 'ERROR = timeout';
  }, MAXIMUM_WAITING_TIME);
</script>
```

COMMAND GUIDE V1.3.8.0

Appendix D – Definition of terms

Term	Definition
boolean	A true or false condition expressed as true false that does not require string "wrapping".*
element	Objects such as buttons used on a User Interface.
external device	Any TCP, Telnet, UDP or Serial device under control.
feedback	Options relating to external device responses.
integer	Decimal number containing 0 – 9 that does not require string "wrapping".*
position	Button element condition of either up or down .
process	Single or combination of commands to obtain a result.
response	String returned by external device in acknowledgment of the received command.
state	Element condition of either enabled or disabled.
string	ASCII text string that requires "wrapping" with quotation marks.
UI	User Interface displayed on a touchscreen.
wrapping	Quotation marks "surrounding" a text string.

* select case () requires any case "<value>" to be wrapped

Appendix E – *Breaking Changes*

Breaking changes to control commands are avoided where possible, but from time to time it is necessary.

Here will be listed impacted commands from which version of software.

Version	Command
1.3.7.0	Split buttons <<Encoder>> replaced with <<encoder_name>>
1.3.7.0	Split buttons <<Decoder>> replaced with <<decoder_name>>